CCIP503 Computer Networks & Security

Week – 3

Application Layer



Assist. Prof. Dr. İlktan AR

0.1 Course – Tentative Outline

Week	Date	Topic
1	28 Sept.	Computer Networks and the Internet
2	5 Oct.	Computer Networks and the Internet
3	12 Oct.	Application Layer
4	26 Oct.	Application Layer
5	2 Nov.	Transport Layer, Wireshark usage and examples
6	9 Nov.	**First Midterm Exam (15:00-17:00 Balat Salon & PC.Lab3)**
7	16 Nov.	Transport Layer
8	23 Nov.	The Network Layer: Data Plane
9	30 Nov.	The Network Layer: Control Plane
10	30 Nov.	The Link Layer and Local Area Networks
11	7 Dec.	**Second Midterm Exam (15:00-17:00 Balat Salon & PC.Lab3)**
12	14 Dec.	Security Technology and Principles
13	21 Dec.	Security Technology and Principles
14	28 Dec.	Security Technology and Principles, Talk for Final Exam
FN	???	**Final Exam**

0.2 Previous Week

- We will focus on Application Layer.



Your duties:

✓ Download and Install WireShark application.

✓ Review this lecture note via learn.khas.edu.tr

✓ Read chapter 1&2 from Kurose & Ross's textbook.


Any Questions

Week 3 - Agenda

01 Principles of Network Applications

02 The Web and HTTP

03 Electronic Mail in the Internet

04 Next Week

1.1 Introduction

- If we couldn't conceive of any useful applications, there wouldn't be any need for networking infrastructure and protocols to support them.

- Internet applications include the classic text-based applications that became popular in the 1970s and 1980s: text e-mail, remote access to computers, file transfers, and newsgroups.

- They include the killer application of the mid-1990s, the World Wide Web, encompassing Web surfing, search, and electronic commerce.

1.2 Introduction (cont.)

- They include instant messaging and P2P file sharing, the two killer applications introduced at the end of the millennium.

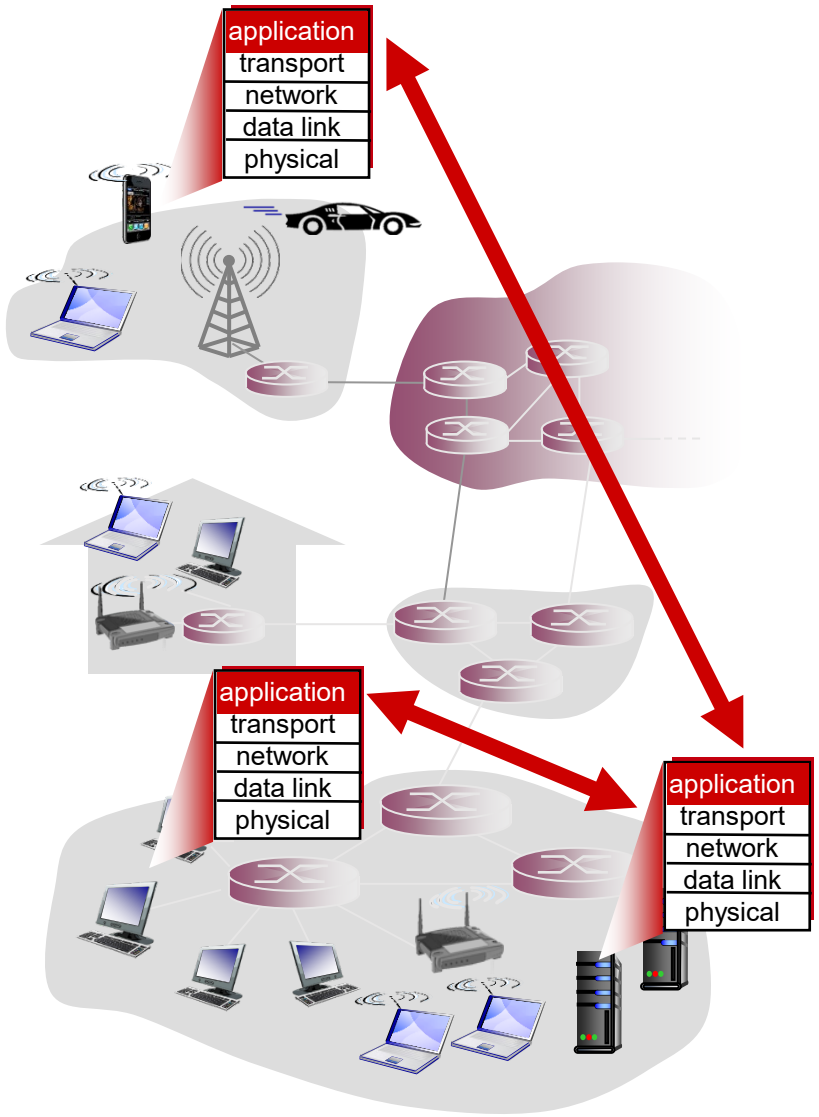
- In the new millennium, new and highly compelling applications continue to emerge, including voice over IP and video conferencing such as Skype, Facetime, and Google Hangouts.

- YouTube, Netflix, multiplayer online games, social networking applications...

1.3 Creating a Network Application

- Write programs that:
  - run on (different) end systems,
  - communicate over network,
  - e.g., web server software communicates with browser software.

- No need to write software for network-core devices:
  - network-core devices do not run user applications,
  - applications on end systems allows for rapid app development, propagation.



1.4 Network Application Architectures

- From the application developer's perspective, the network architecture is fixed and provides a specific set of services to applications. The application architecture, on the other hand, is designed by the application developer and dictates how the application is structured over the various end systems.
- In choosing the application architecture, an application developer will likely draw on one of the two predominant architectural paradigms used in modern network applications: the **client-server architecture** or the **peer-to-peer** (P2P) **architecture**.
- In a client-server architecture, there is an **always-on host**, called the **server**, which services requests from many other hosts, called **client**s. A classic example is the Web application for which an always-on Web server services requests from browsers running on client hosts. When a Web server receives a request for an object from a client host, it responds by sending the requested object to the client host.

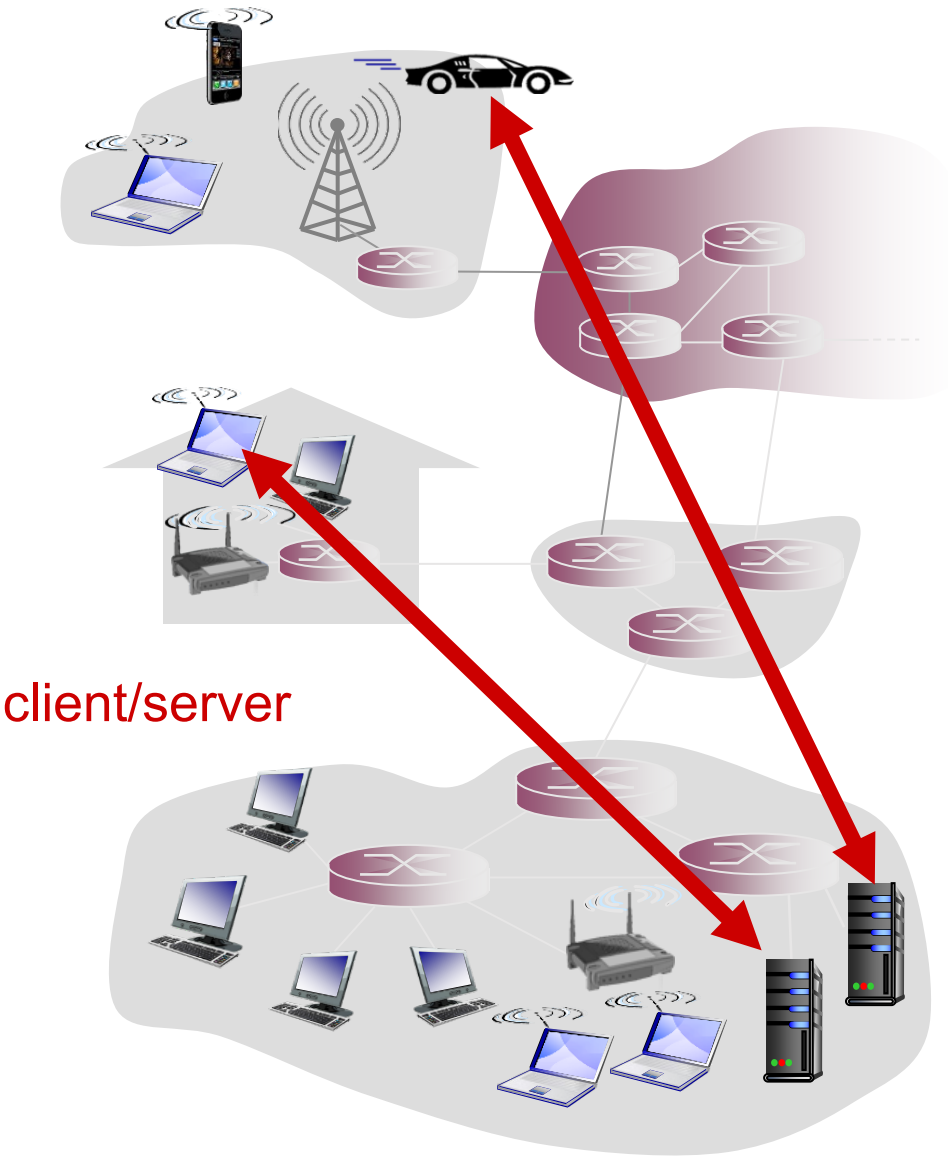
1.5 Network Application Architectures (cont.)

- Often in a client-server application, a single-server host is incapable of keeping up with all the requests from clients. For example, a popular social-networking site can quickly become overwhelmed if it has only one server handling all its requests.
- For this reason, a **data center** is often used to create a powerful virtual server.
- The most popular Internet services-such as search engines (e.g., Google, Bing, Baidu), Internet commerce (e.g., Amazon, eBay, Alibaba), Web based e-mail (e.g., Gmail and Yahoo Mail), social networking (e.g., Facebook, Instagram, Twitter, and WeChat)-employ one or more data centers.
- Google has 30 to 50 data centers **distributed** around the world, which collectively handle search, YouTube, Gmail, and other services.

1.6 Network Application Architectures (cont.)

- Server:
  - always-on host,
  - permanent IP address,
  - data centers for scaling.

- Clients:
  - communicate with server,
  - may be intermittently connected,
  - may have dynamic IP addresses,
  - do not communicate directly with each other.



1.7 Network Application Architectures (cont.)

- In a **P2P architecture**, there is minimal (or no) reliance on dedicated servers in data centers. Instead the application exploits direct communication between pairs of intermittently connected hosts, called **peers**.

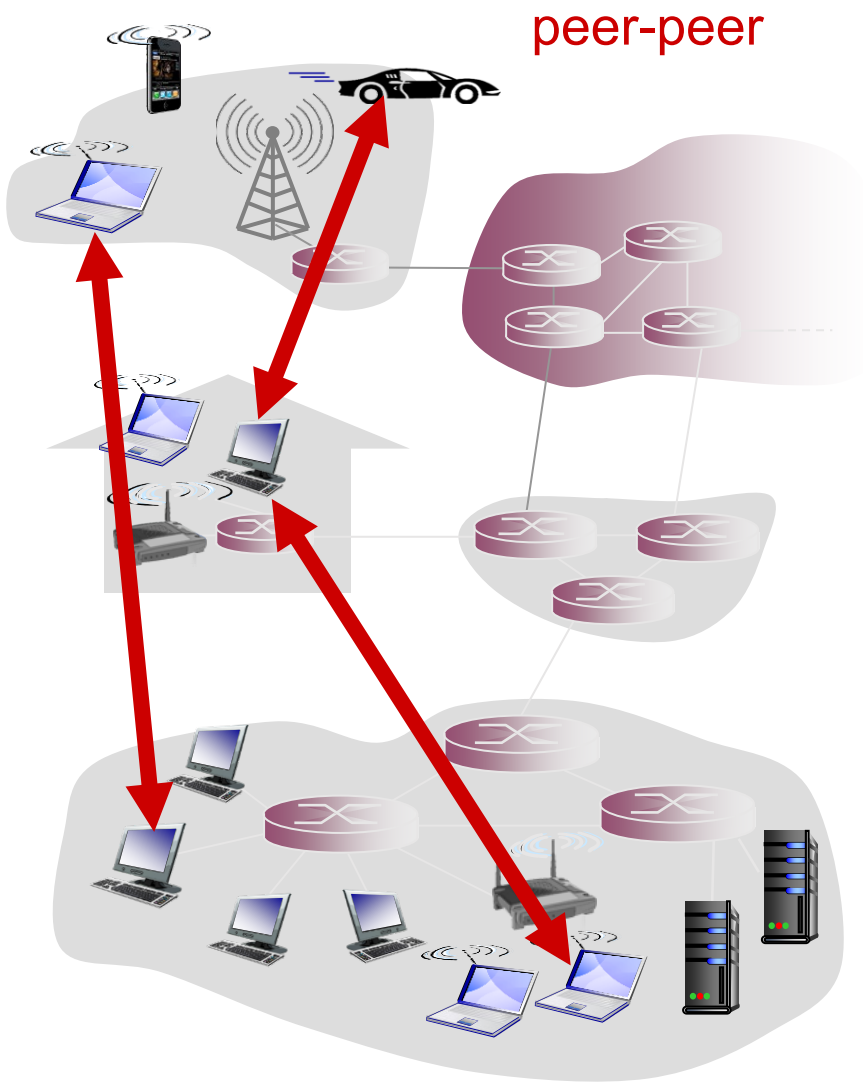
- Because the peers communicate without passing through a dedicated server, the architecture is called **peer-to-peer**.

- Many of today's most popular and traffic-intensive applications are based on P2P architectures. These applications include file sharing (e.g., BitTorrent), peer-assisted download acceleration (e.g., Xunlei), and Internet telephony and video conference (e.g., Skype).

1.8 Network Application Architectures (cont.)

P2P:

- No always-on server,
- Arbitrary end systems directly communicate,
- Peers request service from other peers, provide service in return to other peers
  - **Self scalability** – new peers bring new service capacity, as well as new service demands
- Peers are intermittently connected and change IP addresses
  - complex management

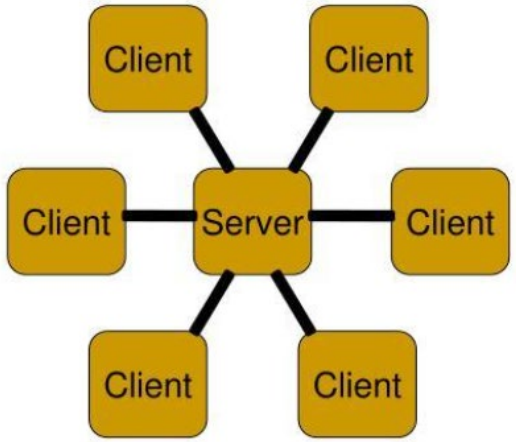


1.9 Application Architecture: Compare the Models

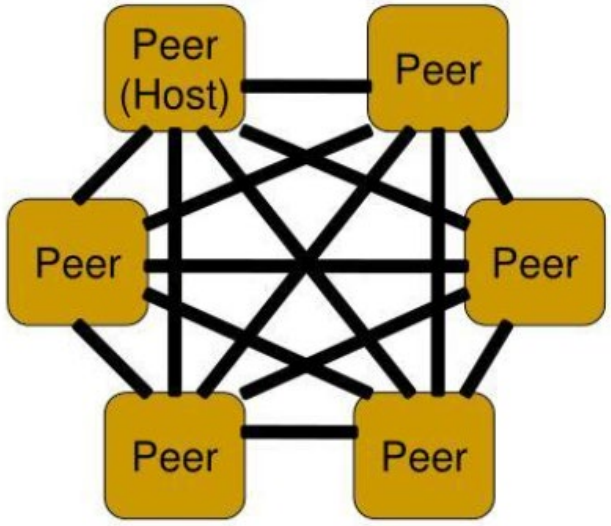
- Client–server: clients request services from a server or server fleet.
- Peer-to-peer (P2P): peers can both request and provide resources.

- Compare: availability, scalability, administration, security, and cost.
- Modern systems are often hybrid: central coordination plus peer-to-peer data exchange.

- *Discussion: Is a video-conferencing application necessarily pure client–server?*



Client-Server Topology



Peer-to-Peer Topology

1.10 Choose an Architecture

- For each service, choose client–server, P2P, or hybrid.

1. Online banking
2. A public BitTorrent distribution
3. A messaging app with central account management and direct media paths
4. A university learning-management system

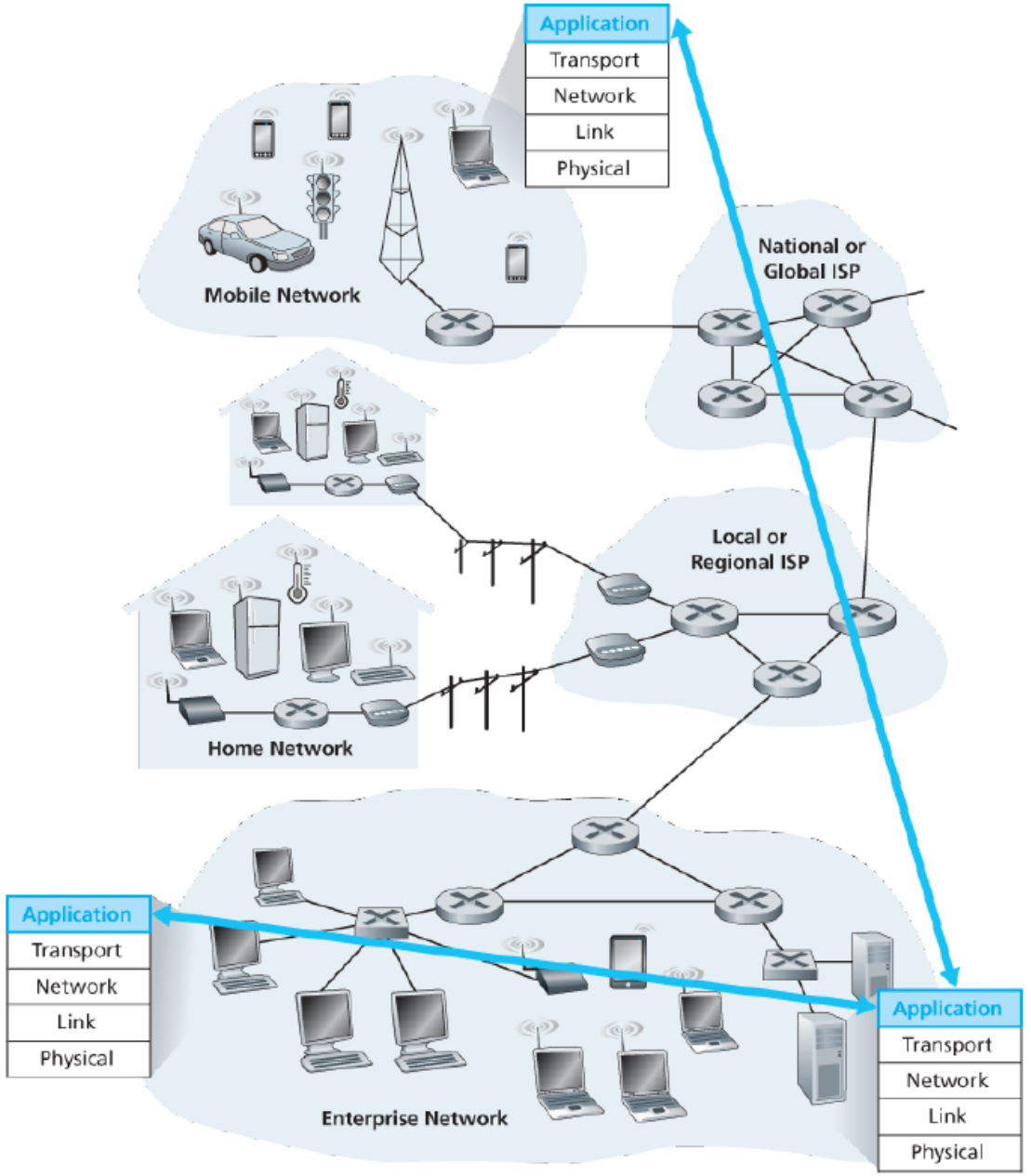
- Client–server, P2P, Hybrid, Client–server

1.11 Processes Communicating

- Before building your network application, you also need a basic understanding of how the programs, running in multiple end systems, communicate with each other.
- In the jargon of operating systems, it is not actually programs but **processes** that communicate.
- When processes are running on the same end system, they can communicate with each other with interprocess communication, using rules that are governed by the end system's operating system. (Nima Hoca's domain ☺)
- Processes on two different end systems communicate with each other by **exchanging messages** across the computer network. A **sending process** creates and sends messages into the network; a **receiving process** receives these messages and possibly responds by sending messages back.

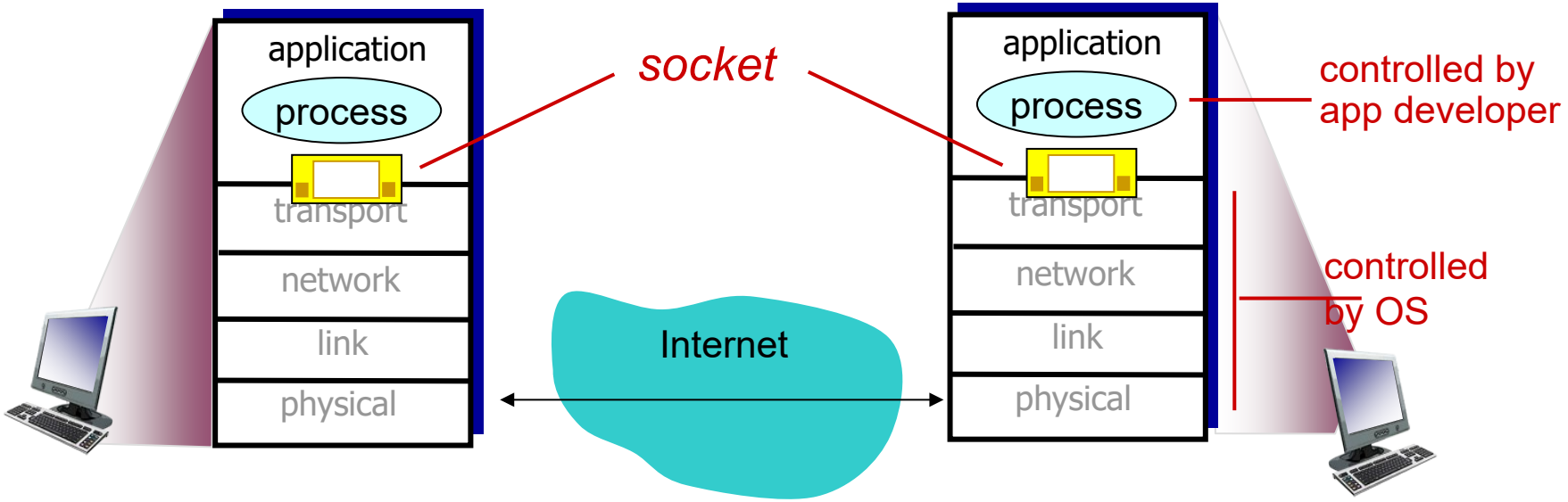
1.12 Processes Communicating

- **Client process**: process that initiates communication
- **Server process**: process that waits to be contacted
- In the Web, a browser process initializes contact with a Web server process; hence the browser process is the client and the Web server process is the server.
- In P2P file sharing, when Peer A asks Peer B to send a specific file, Peer A is the client and Peer B is the server in the context of this specific communication session.



1.13 Socket

- Any message sent from one process to another must go through the underlying network.
- A process sends messages into, and receives messages from, the network through a software interface called a **socket**.
- A process is analogous to a house, and its socket is analogous to its door.
  - sending process shoves message outdoor,
  - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process.



1.14 Addressing Processes

- To send postal mail to a particular destination, the destination needs to have an address.
- Similarly, for a process running on one host to send packets to a process running on another host, the receiving process needs to have an address.

- On the Internet, the host is identified by its **IP address**. (*Further details later, network layer*)
- IP address is a 32-bit quantity that we can think of as uniquely identifying the host.

1.15 Addressing Processes (cont.)

- Does IP address of host on which process runs suffice for identifying the process?
  - No, many processes can be running on same host !!

- The sending process must also identify the receiving process (receiving socket) running in the host. A destination **port number** serves this purpose. Popular applications have been assigned specific port numbers. HTTP server: 80, mail server: 25
- To send HTTP message to gaia.cs.umass.edu web server: IP address: 128.119.245.12, port number: 80

1.16 Application Layer Protocol Defines:

- types of messages exchanged,
  - e.g., request, response
- message syntax:
  - what fields in messages & how fields are delineated
- message semantics
  - meaning of information in fields
- rules for when and how processes send & respond to messages

open protocols:

- defined in RFCs
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype

1.17 What Transport Service does an App Need?

- Recall that a socket is the interface between the application process and the transport-layer protocol.
- The application at the sending side pushes messages through the socket. At the other side of the socket, the transport-layer protocol has the responsibility of getting the messages to the socket of the receiving process.
- Many networks, including the Internet, provide more than one transport-layer protocol.
- The situation is like choosing either train or airplane transport for travel between two cities. You must choose one or the other, and each transportation mode offers different services.
- We can broadly classify the possible services along four dimensions: reliable data transfer, throughput, timing, and security.

1.18 What Transport Service does an App Need?

- Data integrity:
  - some apps (e.g., file transfer, web transactions) require 100% reliable data transfer,
  - other apps (e.g., audio) can tolerate some loss.
- Throughput:
  - some apps (e.g., multimedia) require minimum amount of throughput to be “effective”,
  - other apps (“elastic apps”) make use of whatever throughput they get.
- Timing:
  - some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”
- Security:
  - encryption, data integrity, …

1.19 Transport Services Provided by the Internet

- We have been considering transport services that a computer network could provide in general.
- The Internet (and, more generally, TCP/IP networks) makes two transport protocols available to applications, **UDP** and **TCP**.
- When you (as an application developer) create a new network application for the Internet, one of the first decisions you have to make is whether to use UDP or TCP.

1.20 Transmission Control Protocol (TCP)

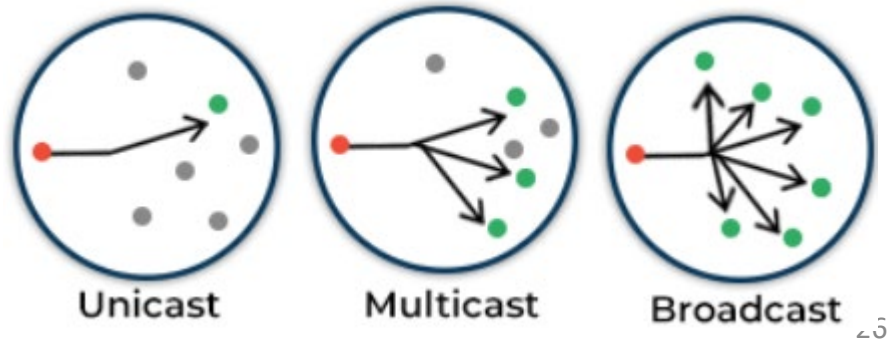
- The TCP service model includes a **connection-oriented service** and a reliable data transfer service. After a handshaking phase, a **TCP connection** is said to exist between the sockets of the two processes. The connection is a full-duplex connection in that the two processes can send messages to each other over the connection at the same time.

- **Reliable data transfer service.** The communicating processes can rely on TCP to deliver all data sent without error and in the proper order.

- TCP also includes a **congestion-control** mechanism that throttles a sending process (client or server) when the network is congested between sender and receiver.

1.21 User Datagram Protocol (UDP)

- UDP is connectionless, so there is no handshaking before the two processes start to communicate.
- UDP provides an unreliable data transfer service—that is, when a process sends a message into a UDP socket, UDP provides no guarantee that the message will ever reach the receiving process.
- Furthermore, messages that do arrive at the receiving process may arrive out of order.
- UDP does not include a congestion-control mechanism, so the sending side of UDP can pump data into the layer below (the network layer) at any rate it pleases.
- Why bother? Why is there a UDP? ***But UDP is more handsome/pretty ☺, he/she loves me...***



1.22 TCP-UDP

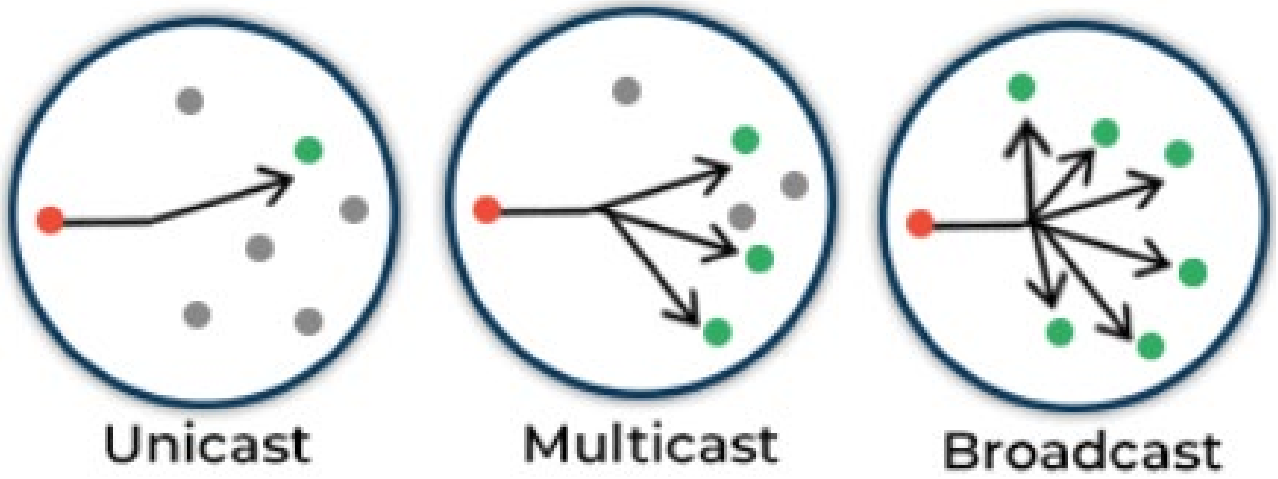


- TCP is slow, but complete data delivery.
- UDP is fast, but at risk of incomplete data delivery. Typical applications of UDP: Live streaming, Online Games, VoIP.
- Tcp support only unicast, UDP supports all type of casts.

1.23 Unicast, Multicast, Broadcast…



- Tcp support only unicast, UDP supports all type of casts.



1.24 Internet Apps: Application, Transport Protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP 1.1 [RFC 7320]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7320], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

1.25 TCP and UDP: A More Precise Comparison

- TCP: connection-oriented byte stream; reliable, ordered delivery; congestion control.
- UDP: connectionless datagrams; no built-in delivery, ordering, or retransmission guarantee.

- Neither protocol is universally “faster”: performance depends on the application and network.
- Applications can add reliability or congestion control above UDP when needed.

Week 3 - Agenda

01 Principles of Network Applications

02 The Web and HTTP

03 Electronic Mail in the Internet

04 Next Week

2.1 Introduction

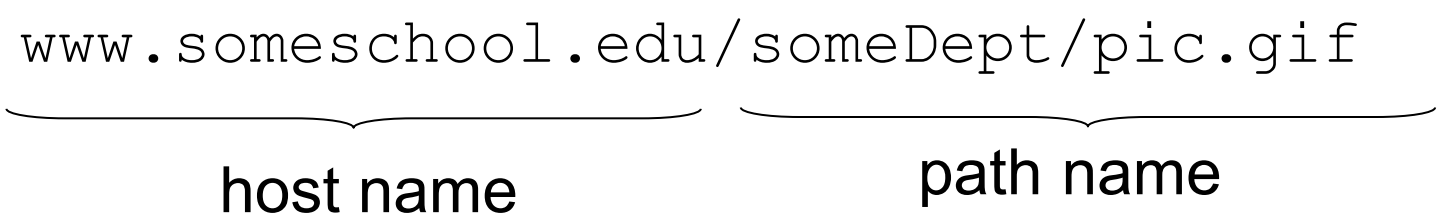
- Until the early 1990s the Internet was used primarily by researchers, academics, and university students to log in to remote hosts, to transfer files from local hosts to remote hosts and vice versa, to receive and send news, and to receive and send electronic mail.

- In the early 1990s, a major new application arrived on the scene—the **World Wide Web** [Berners-Lee 1994]. The Web was the first Internet application that caught the general public's eye.

- Perhaps what appeals the most to users is that the Web operates **on demand**. **Users receive what they want, when they want it**. This is unlike traditional broadcast radio and television, which force users to tune in when the content provider makes the content available.

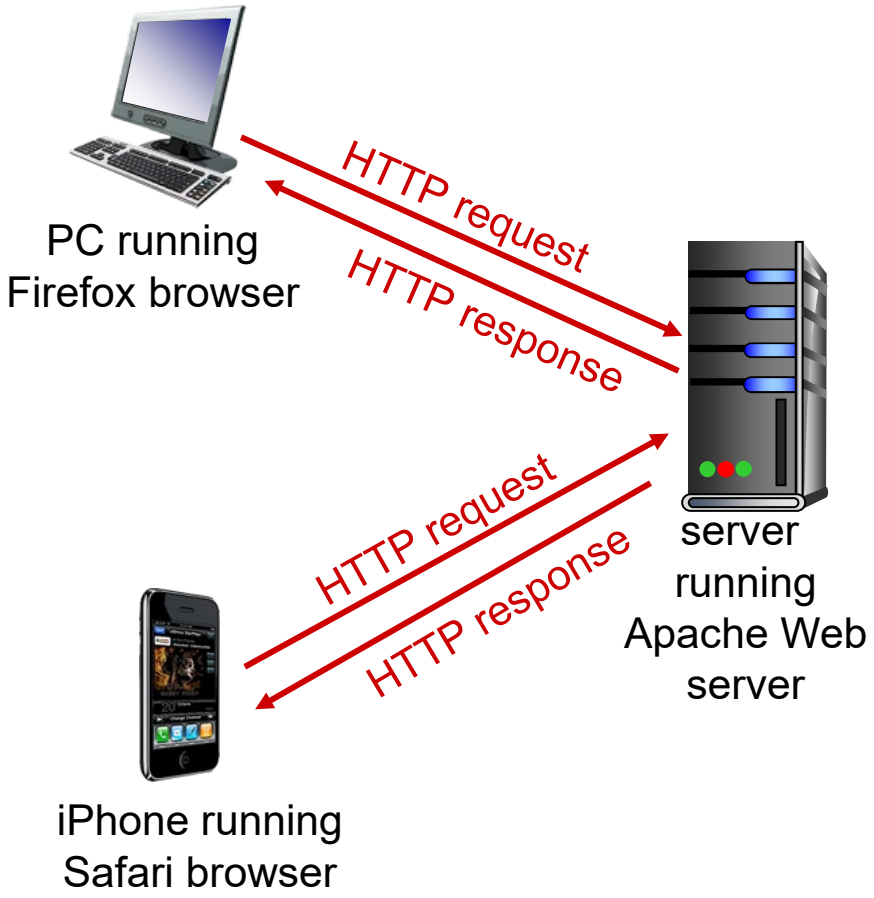
2.2 Overview of HTTP

- The **HyperText Transfer Protocol** (HTTP), the Web's application-layer protocol, is at the heart of the Web.
- HTTP is implemented in two programs: a client program and a server program.
- The client program and server program, executing on different end systems, talk to each other by exchanging HTTP messages. HTTP defines the structure of these messages and how the client and server exchange the messages.
- Web page consists of objects which can be HTML file, JPEG image, Java applet, audio file,…
- Web page consists of base HTML-file which includes several referenced objects. Each object is addressable by a URL (Uniform Resource Locator), e.g.,



2.3 Overview of HTTP (cont.)

- HTTP: hypertext transfer protocol
- Web's application layer protocol
- Client/Server model
  - client: browser that requests, receives, (using HTTP protocol) and "displays" Web objects
  - server: Web server sends (using HTTP protocol) objects in response to requests



2.4 Overview of HTTP (cont.)

- **Uses TCP**:
  - Client initiates TCP connection (creates socket) to server, port 80.
  - Server accepts TCP connection from client.
  - HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server).
  - TCP connection closed.
- **HTTP is "stateless" protocol**: server maintains no information about past client requests.
- If a particular client asks for the same object twice in a period of a few seconds, the server does not respond by saying that it just served the object to the client; instead, the server resends the object, as it has completely forgotten what it did earlier.

2.5 HTTP Connections

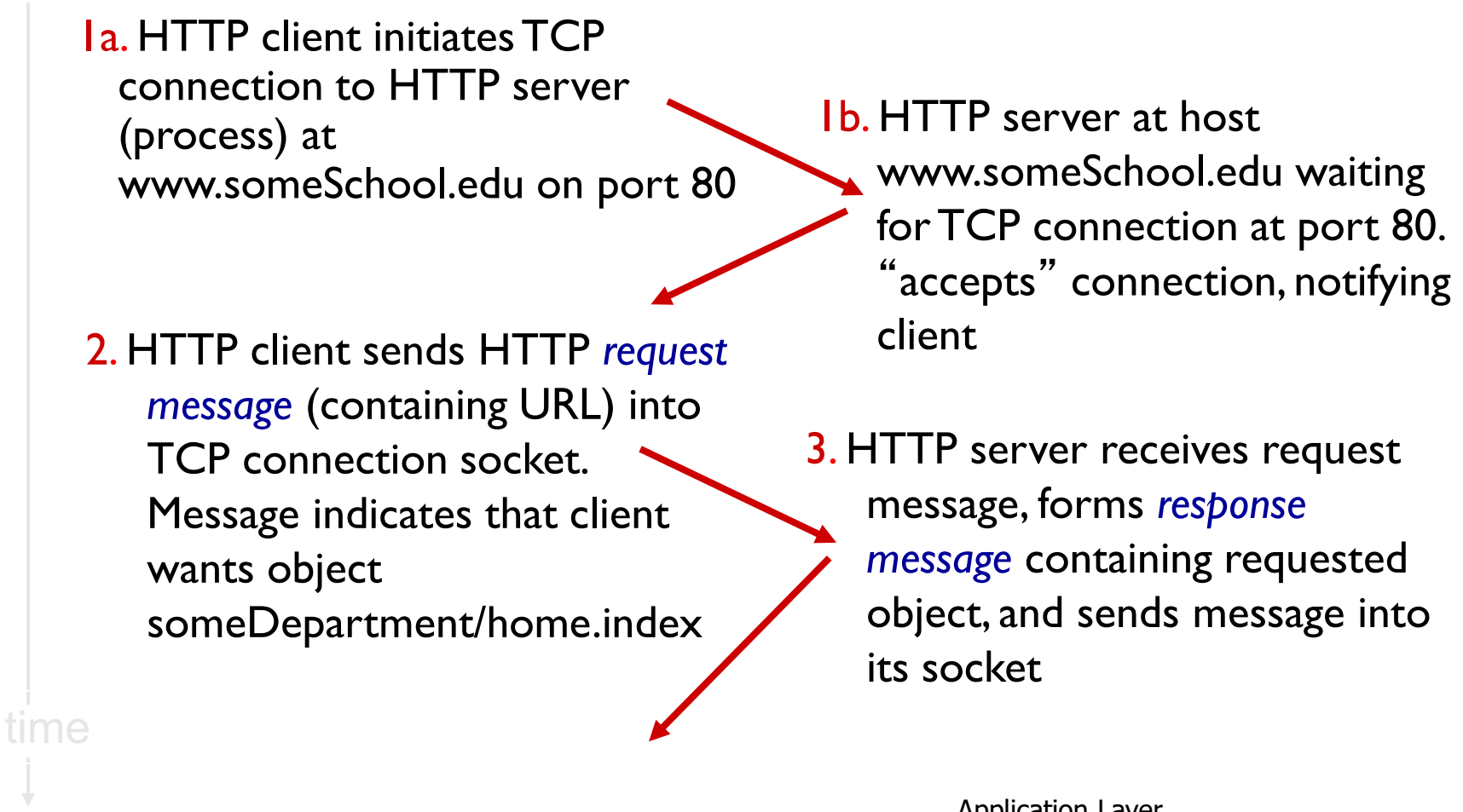
- When the client-server interaction is taking place over TCP, the application developer needs to make an important decision:
  - **Non-persistent connections**: Should each request/response pair be sent over a separate TCP connection,
  - **Persistent connections**: Should all of the requests and their corresponding responses be sent over the same TCP connection?

- **Non-persistent connections**: at most one object sent over TCP connection, connection then closed, downloading multiple objects required multiple connections.
- **Persistent connections**: multiple objects can be sent over single TCP connection between client, server.

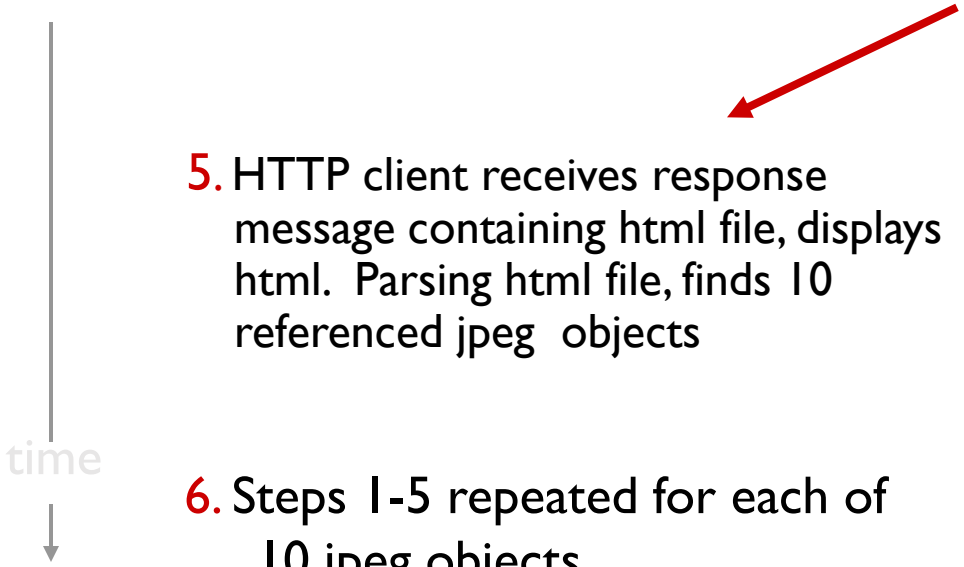
2.6 Non-persistent HTTP

suppose user enters URL:
`www.someSchool.edu/someDepartment/home.index`

(contains text, references to 10 jpeg images)



2.7 Non-persistent HTTP (cont.)



4. HTTP server closes TCP connection.

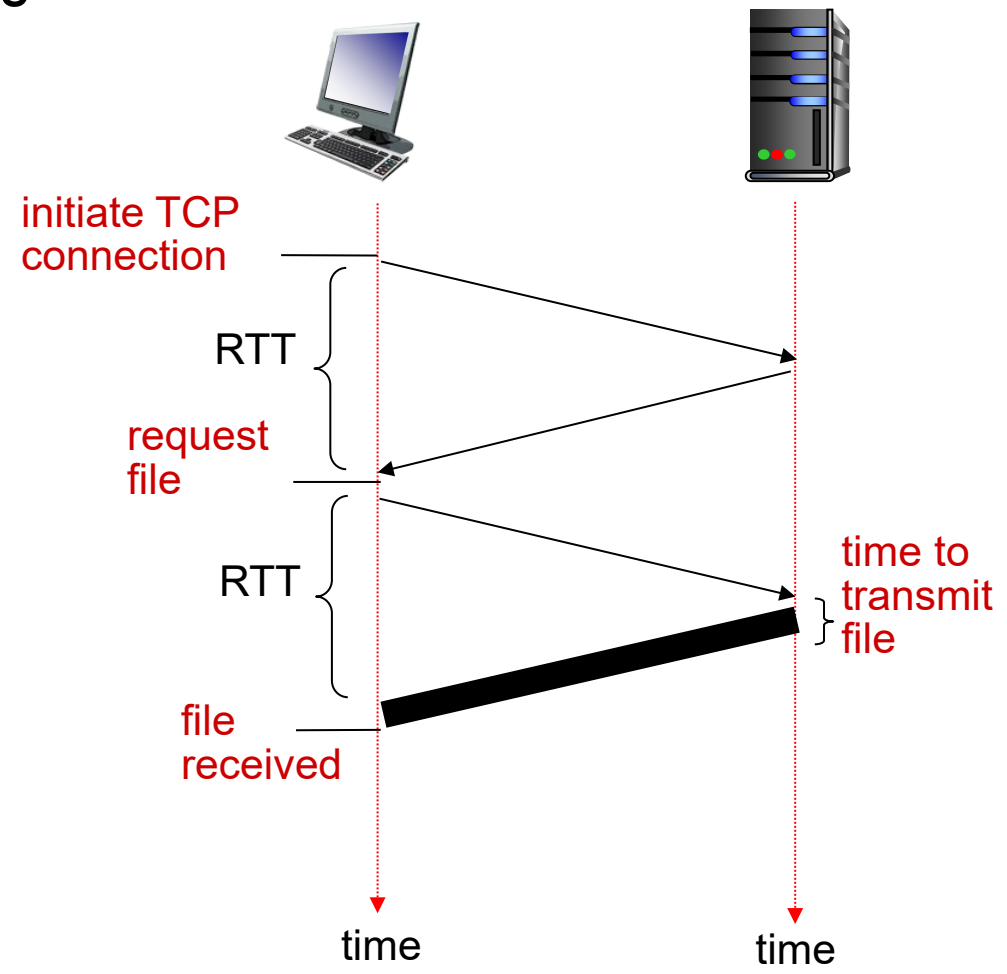
5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

2.8 Non-persistent HTTP: Response Time

- **The round-trip time (RTT)**, which is the time it takes for a small packet to travel from client to server and then back to the client.
- HTTP response time:
  - one RTT to initiate TCP connection,
  - one RTT for HTTP request and first few bytes of HTTP response to return,
  - file transmission time.
- non-persistent HTTP response time =

  2RTT+ file transmission time



The RTT includes packet-propagation delays, packet queuing delays in intermediate routers and switches, and packet-processing delays.

2.9 Persistent HTTP

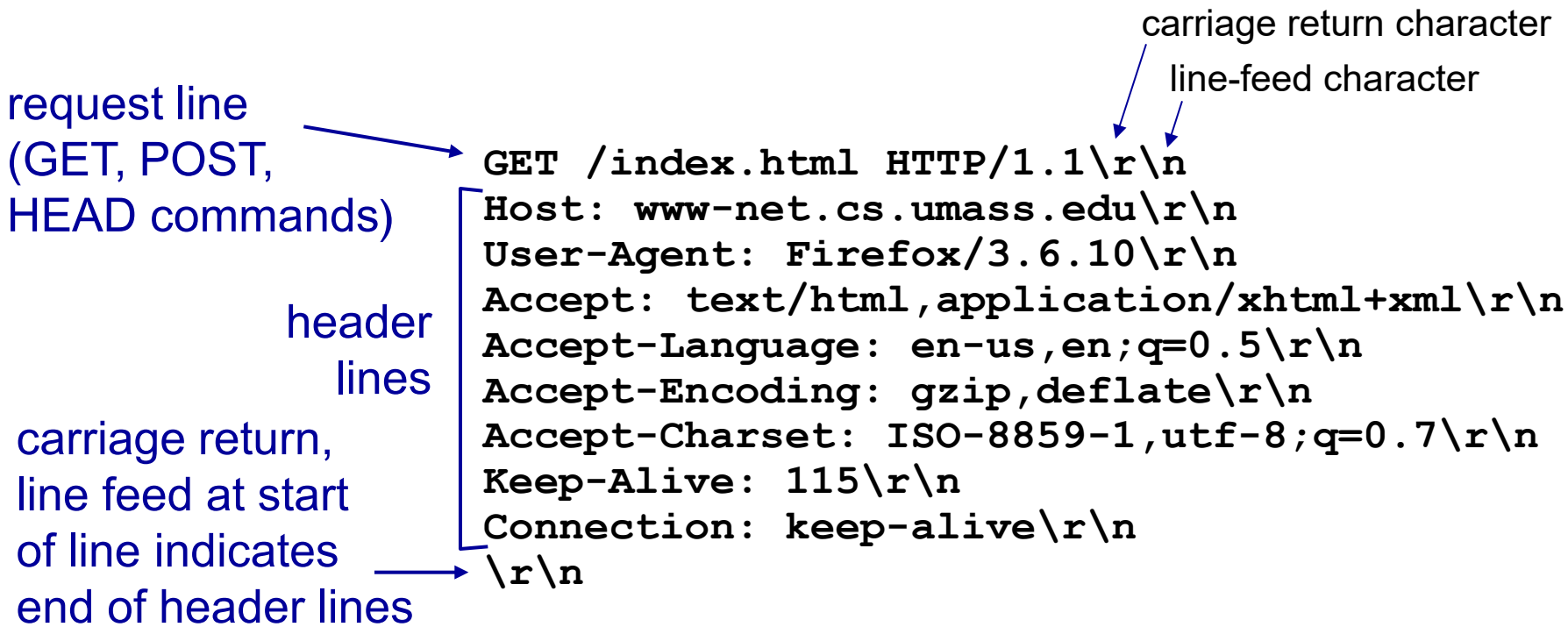
- With HTTP 1.1 persistent connections, the server leaves the TCP connection open after sending a response.
- Subsequent requests and responses between the same client and server can be sent over the same connection. An entire Web page (in the previous example, the base HTML file and the 10 images) can be sent over a single persistent TCP connection.
- Moreover, multiple Web pages residing on the same server can be sent from the server to the same client over a single persistent TCP connection.
- These requests for objects can be made back-to-back, without waiting for replies to pending requests (pipelining).
- Typically, the HTTP server closes a connection when it isn't used for a certain time (a configurable timeout interval).
- As little as one RTT for all the referenced objects.


10 mins break,
QR attendance!
TIME
FOR A
BREAK!

2.10 HTTP Request Message

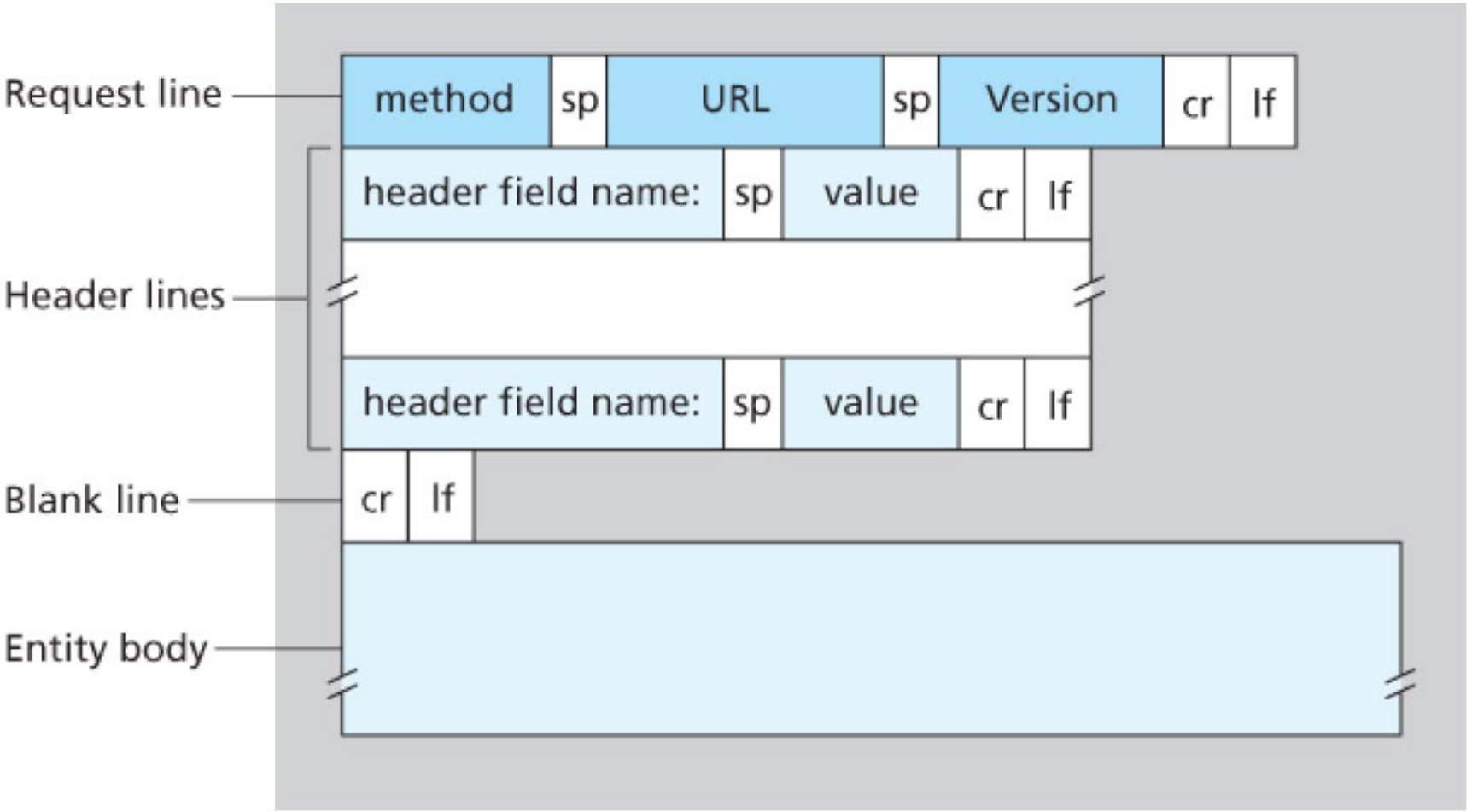
- There are two types of HTTP messages: request, response
- HTTP request message is in ASCII (human-readable format) form.
- By including the *Connection: keep-alive* header line, the browser is telling the server that it wants persistent connections.

When a server receives a request with the *HEAD* method, it responds with an HTTP message, but it leaves out the requested object.



2.11 General Format of an HTTP Request Message

- The entity body is empty with the GET method but is used with the POST method.
- An HTTP client often uses the POST method when the user fills out a form: for example, when a user provides search words to a search engine.
- For example, if a form uses the GET method, has two fields, and the inputs to the two fields are monkeys and bananas , then the URL will have the structure www.somesite.com/animalsearch?monkeys&bananas.



2.12 Method Types

- **HTTP/1.0**:
  - GET
  - POST
  - HEAD
    - asks server to leave requested object out of response

- **HTTP/1.1**:
- GET, POST, HEAD
- PUT
  - uploads file in entity body to path specified in URL field
- DELETE
  - deletes file specified in the URL field

2.13 HTTP Methods

- GET: retrieve a representation; should not be used to request a state-changing action.
- HEAD: like GET but returns headers without the representation body.
- POST: submit data for processing; effects depend on the application.
- PUT: create/replace the target resource representation; defined as idempotent.
- DELETE: request removal of a target resource; defined as idempotent, though repeated responses can differ.

2.14 HTTP Response Message

status line (protocol status code status phrase)

header lines

data, e.g., requested HTML file

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
data data data data data ...
```

2.15 General Format of an HTTP Response Message

STATUS CODES:

200 OK

- request succeeded, requested object later in this msg

301 Moved Permanently

- requested object moved, new location specified later in this msg (Location:)

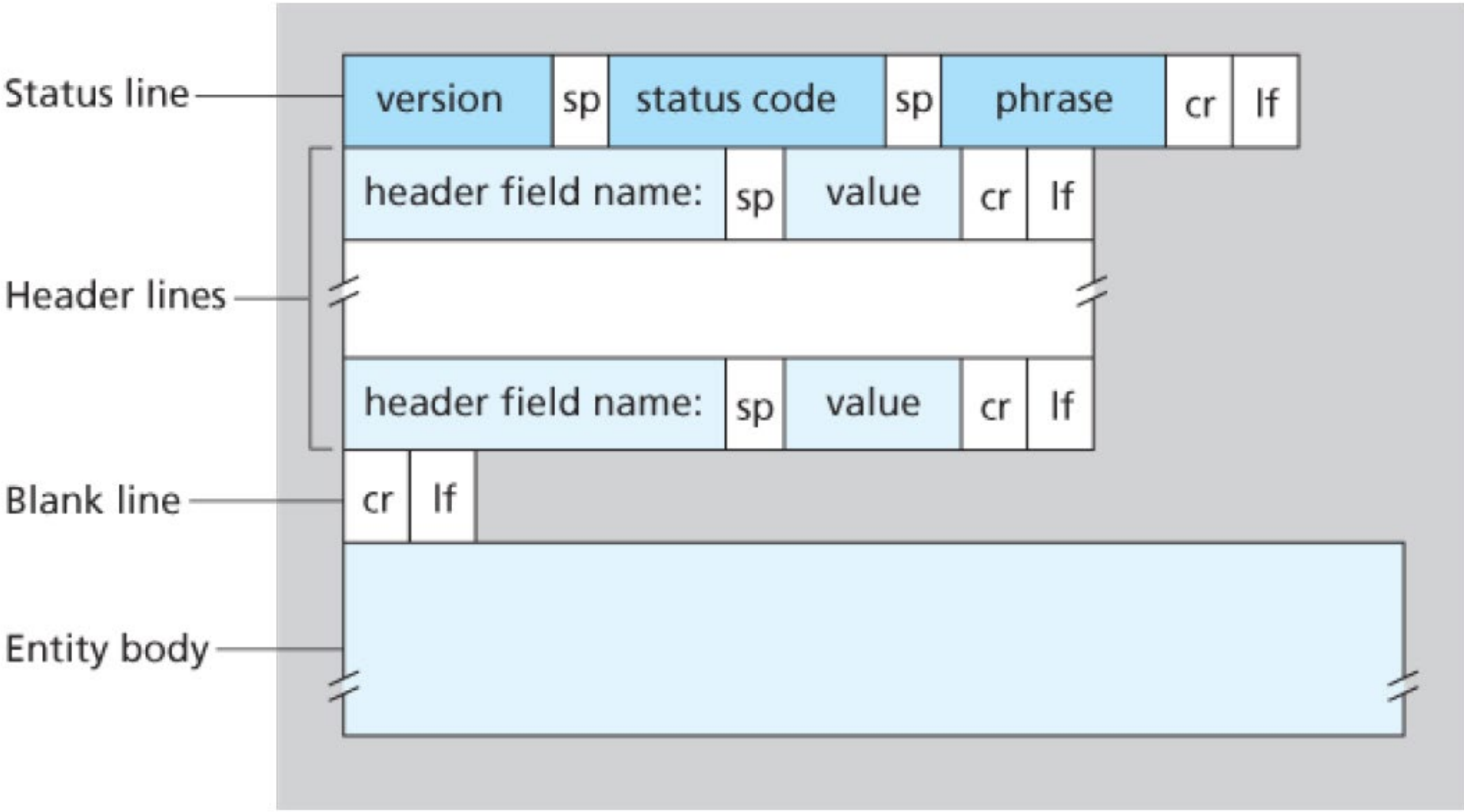
400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported



2.16 Diagnose the Status

- A browser receives each response. Suggest the likely next step:
- 304 Not Modified
  - The browser reuses its cached copy of the resource…
- 401 Unauthorized
  - Retry the request with valid credentials…
- 403 Forbidden
  - User may lack permission
- 429 Too Many Requests
  - Retry with an appropriate delay.
- 503 Service Unavailable
  - The server is temporarily unable to handle the request…

2.17 User-Server Interaction: Cookies



- We mentioned before that an HTTP server is stateless.
- This simplifies server design and has permitted engineers to develop high-performance Web servers that can handle thousands of simultaneous TCP connections.
- However, it is often desirable for a Web site to identify users, either because the server wishes to restrict user access or because it wants to serve content as a function of the user identity. For these purposes, HTTP uses **cookies**.
- Cookies, defined in [RFC 6265], allow sites to keep track of users. Most major commercial Web sites use cookies today.

2.18 User-Server Interaction: Cookies (cont.)

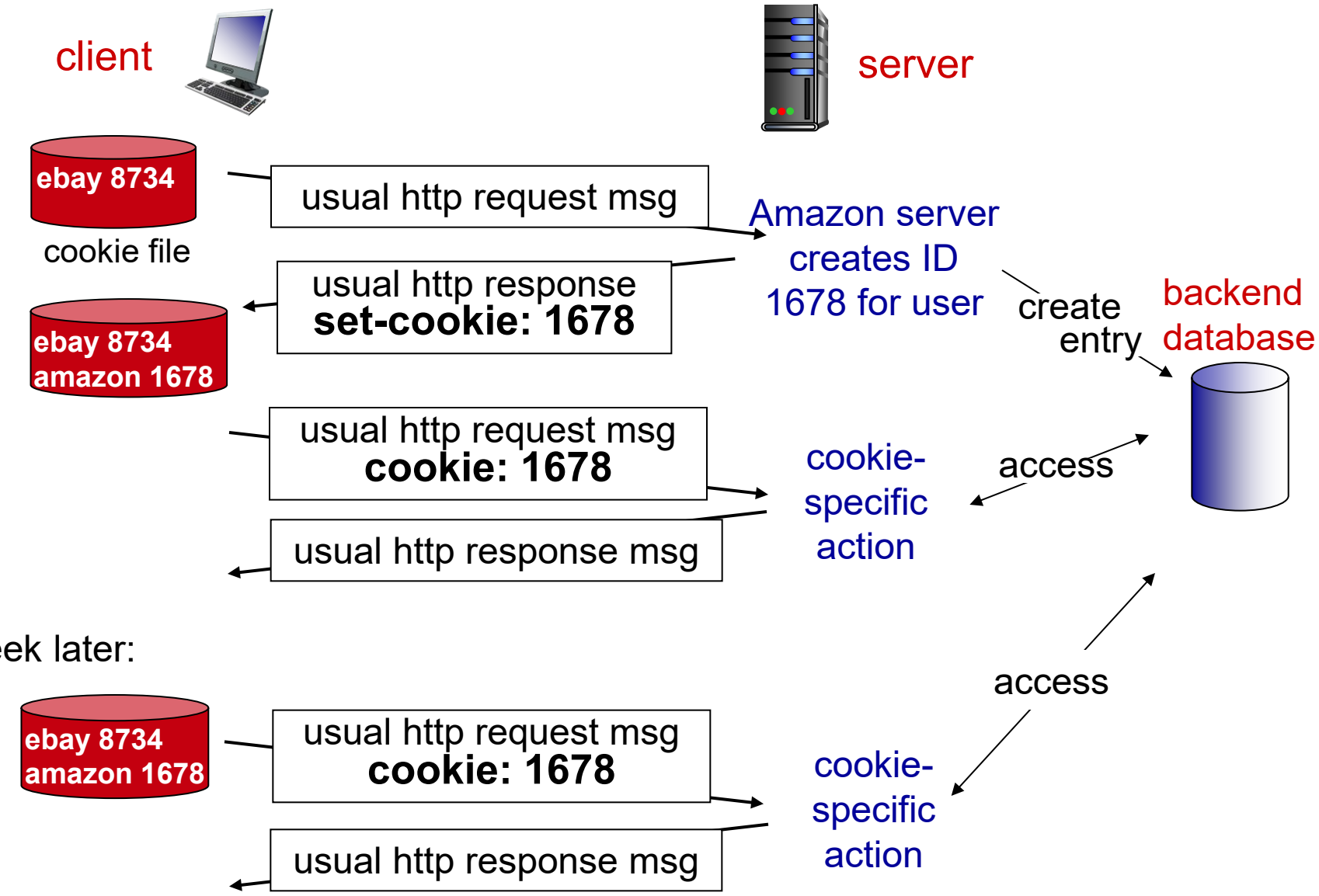
- Cookie technology has four components:

(1) a cookie header line in the HTTP response message;

(2) a cookie header line in the HTTP request message;

(3) a cookie file kept on the user's end system and managed by the user's browser;

(4) and a back-end database at the Web site.



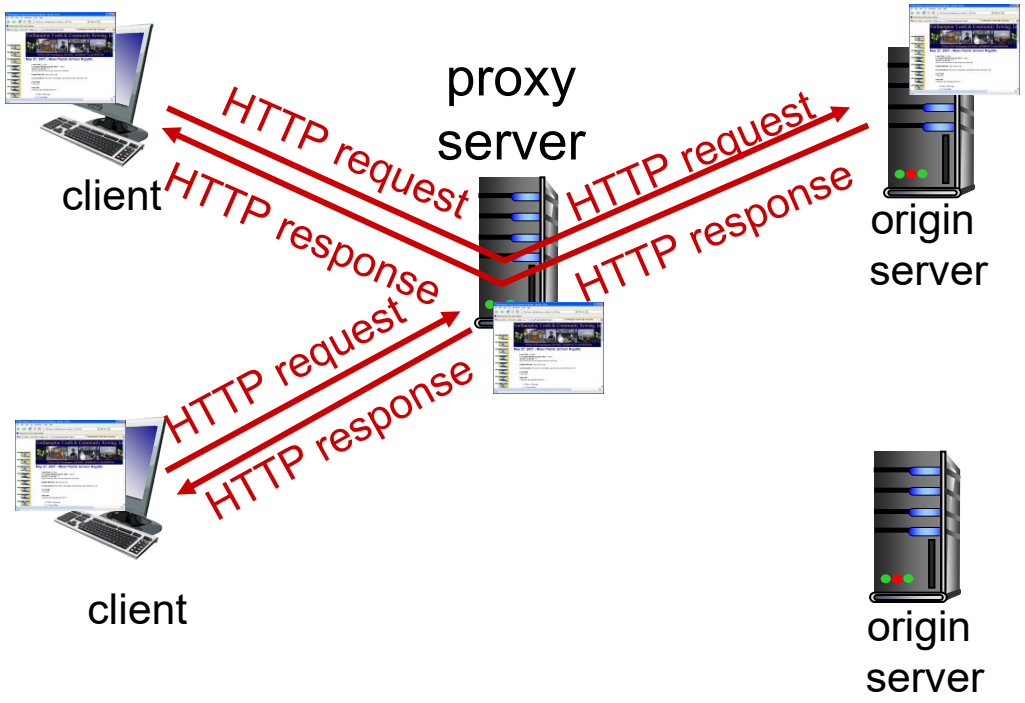
2.19 User-Server Interaction: Cookies (cont.)

- If Susan also registers herself with Amazon-providing full name, e-mail address, postal address, and credit card information-Amazon can then include this information in its database, thereby associating Susan's name with her identification number (and all of the pages she has visited at the site in the past!).
- This is how Amazon and other e-commerce sites provide "one-click shopping" when Susan chooses to purchase an item during a subsequent visit, she doesn't need to re-enter her name, credit card number, or address.
- What cookies can be used for: authorization, shopping carts, recommendations, user session state (Web e-mail).
- How to keep "state": protocol endpoints: maintain state at sender/receiver over multiple transactions, cookies: http messages carry state.



2.20 Web Caching

- A **Web cache**-also called a **proxy server**-is a network entity that satisfies HTTP requests on the behalf of an origin Web server.
- The Web cache has its own disk storage and keeps copies of recently requested objects in this storage.
- The goal is to satisfy client request without involving origin.
- User sets browser: Web accesses via cache,
- Browser sends all HTTP requests to cache;
  - object in cache: cache returns object
  - else cache requests object from origin server, then returns object to client



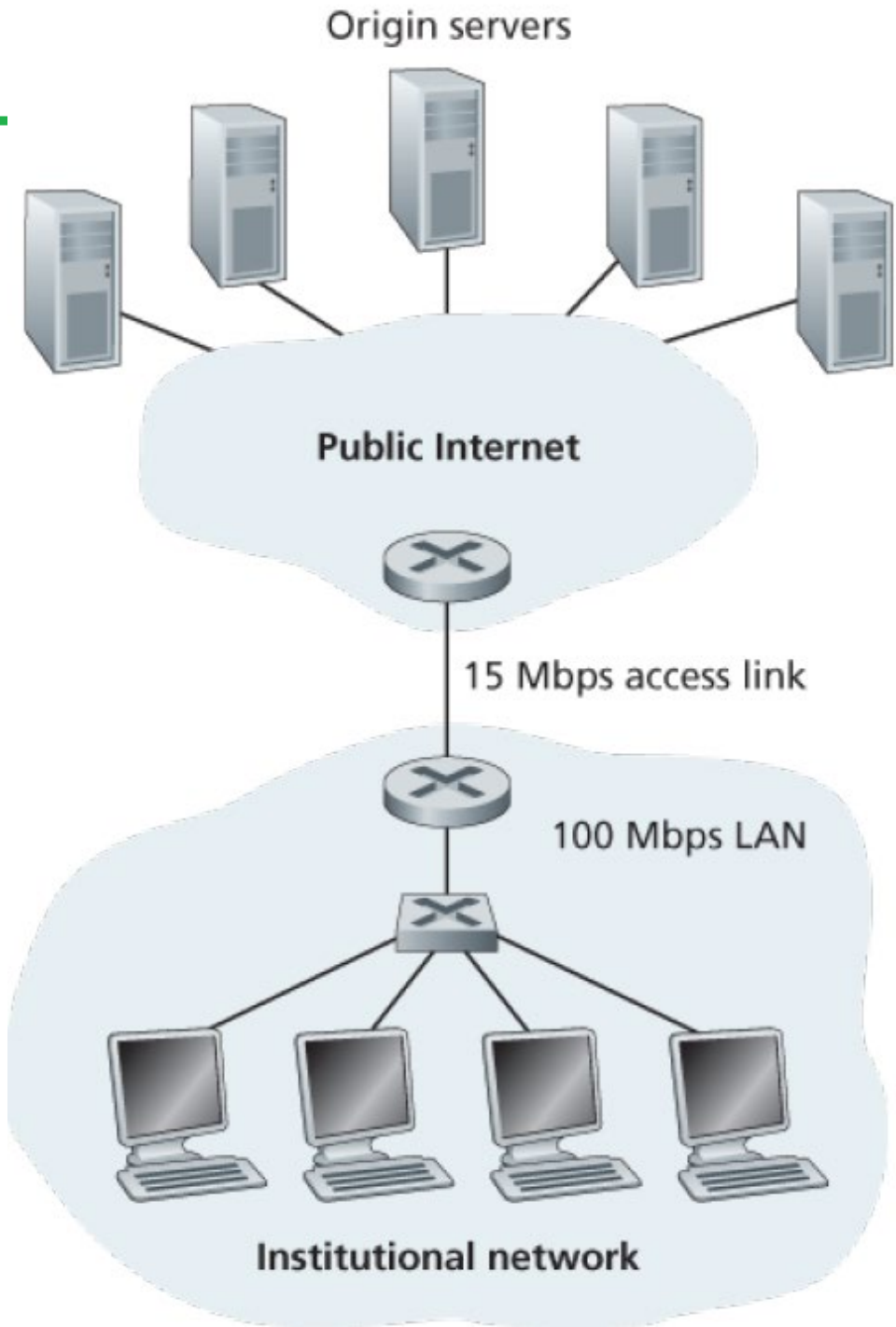
2.21 More about Web Caching

- Cache acts as both client and server:
  - server for original requesting client
  - client to origin server
- Typically cache is installed by ISP (university, company, residential ISP)
- Reduce response time for client request
- Reduce traffic on an institution's access link
- Internet dense with caches enables "poor" content providers to effectively deliver content.

2.22 Example Question

- The institutional network is a high-speed LAN.
- A router in the institutional network and a router on the Internet are connected by a 15 Mbps link.
- avg object size: 1M bits
- avg request rate from browsers to origin servers:15/sec
- RTT from institutional router to any origin server: 2 sec
- So the traffic intensity on the LAN is (15 requests/sec)·(1 Mbits/request)/(100 Mbps) = **0.15** [tens of milliseconds of delay]
- Whereas the traffic intensity on the access link (from the Internet router to institution router) is (15 requests/sec)·(1 Mbits/request)/(15 Mbps) = 1 [the delay on a link becomes very large and grows without bound]
- total delay = Internet delay + access delay + LAN delay

= 2 sec + minutes + millisecs

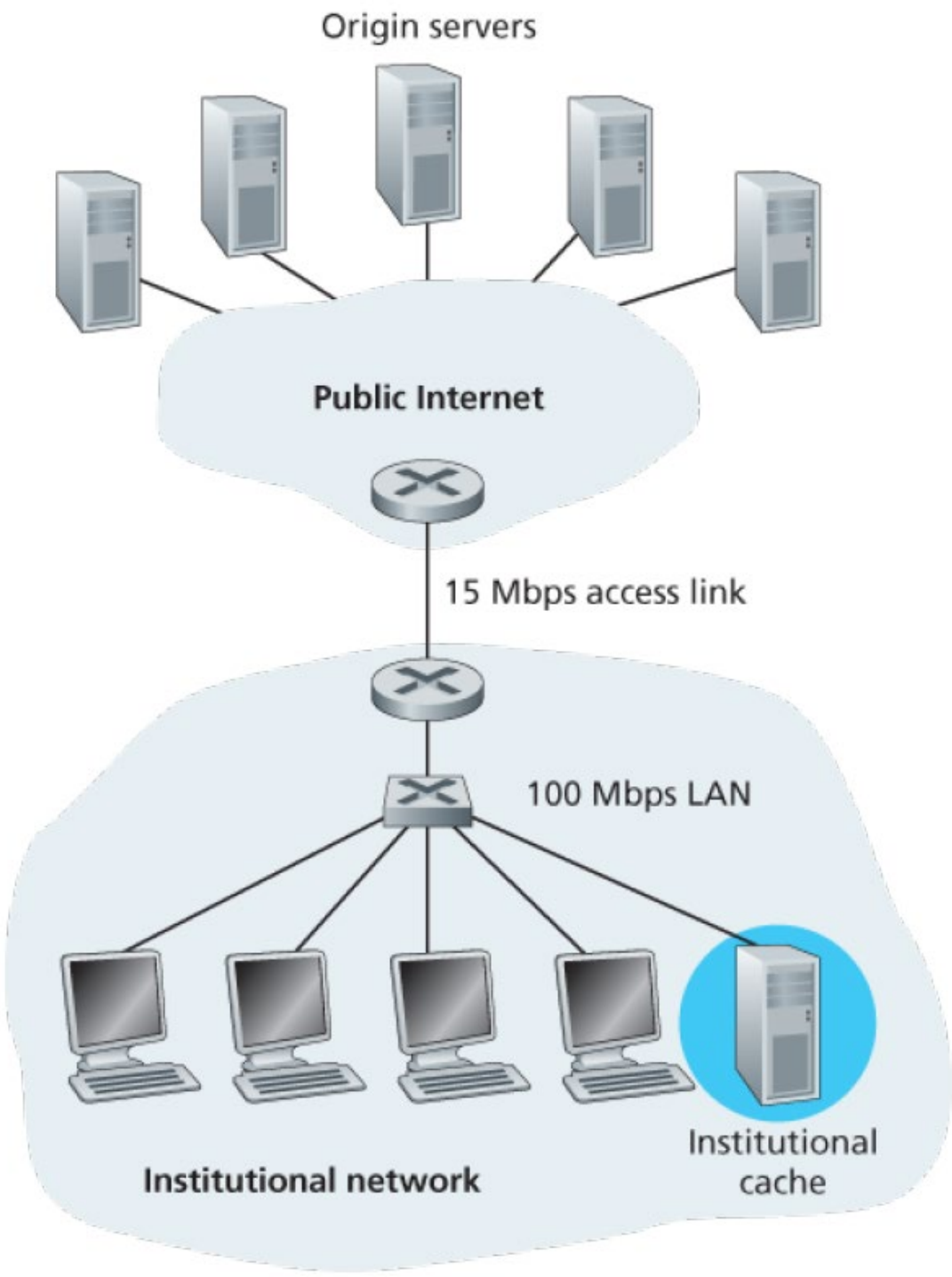


2.23 Solution 1

- One possible solution is to increase the access rate from 15 Mbps to, say, 100 Mbps.
- This will lower the traffic intensity on the access link to 0.15, which translates to negligible delays between the two routers.
- In this case, the total response time will roughly be two seconds, that is, the Internet delay.
- But this solution also means that the institution must upgrade its access link from 15 Mbps to 100 Mbps, a costly proposition.

2.24 Solution 2

- Installing a Web cache in the institutional network.
- Let's suppose that the cache provides a hit rate of 0.4 for this institution. Because the clients and the cache are connected to the same high-speed LAN, 40 percent of the requests will be satisfied almost immediately, say, within 10 milliseconds, by the cache.
- 60 percent of the requested objects passing through the access link, the traffic intensity on the access link is reduced from **1.0** to **0.6**. Typically, a traffic intensity less than 0.8 corresponds to a small delay, say, tens of milliseconds, on a 15 Mbps link.
- Average delay therefore is 0.4·(0.01 seconds)+0.6·(2.01 seconds) which is just slightly greater than 1.2 seconds.

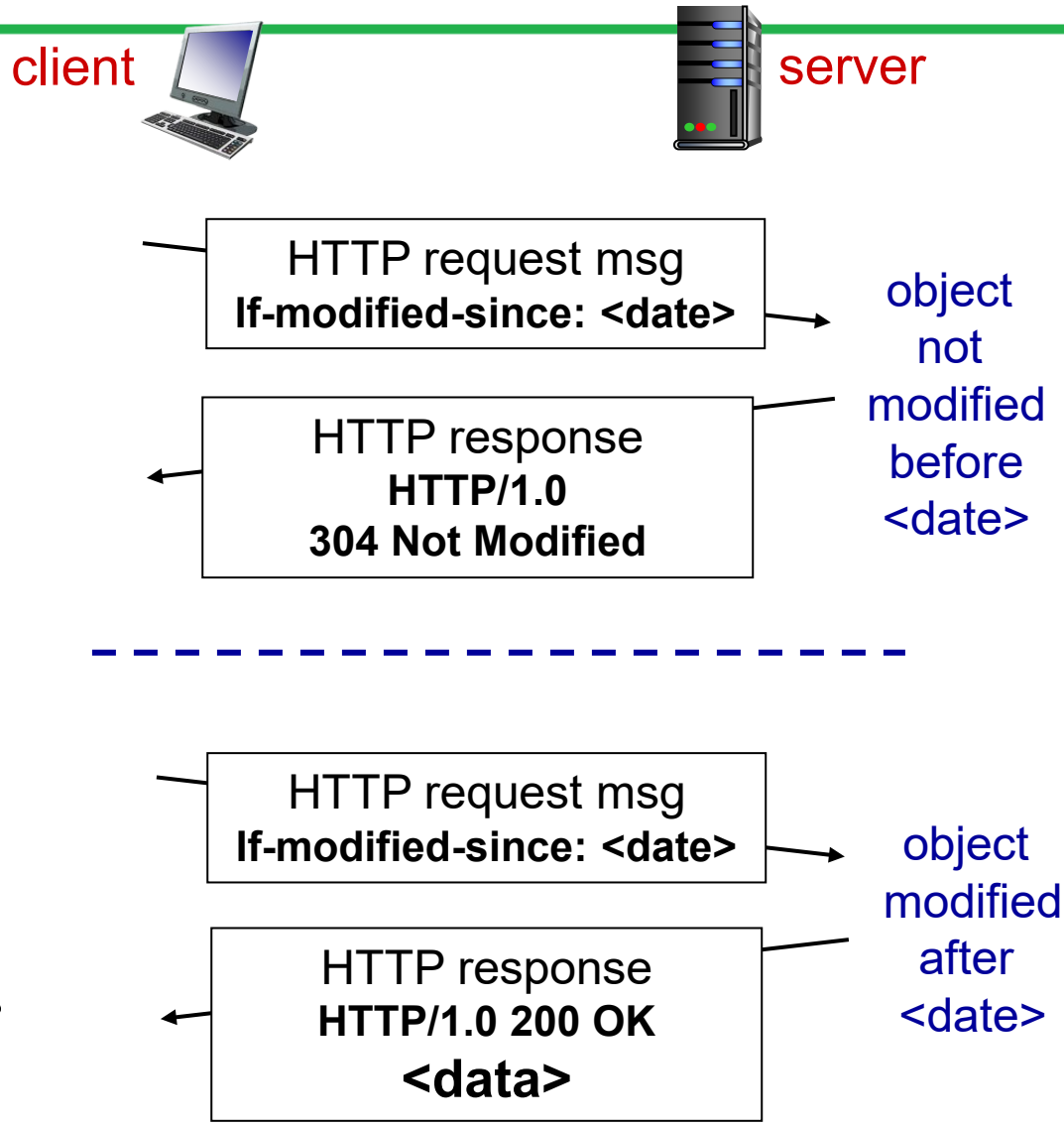


2.25 Conditional GET

- *Goal:* don't send object if cache has up-to-date cached version
  - no object transmission delay
  - lower link utilization
- *cache:* specify date of cached copy in HTTP request

  `If-modified-since: <date>`
- *server:* response contains no object if cached copy is up-to-date:

  `HTTP/1.0 304 Not Modified`

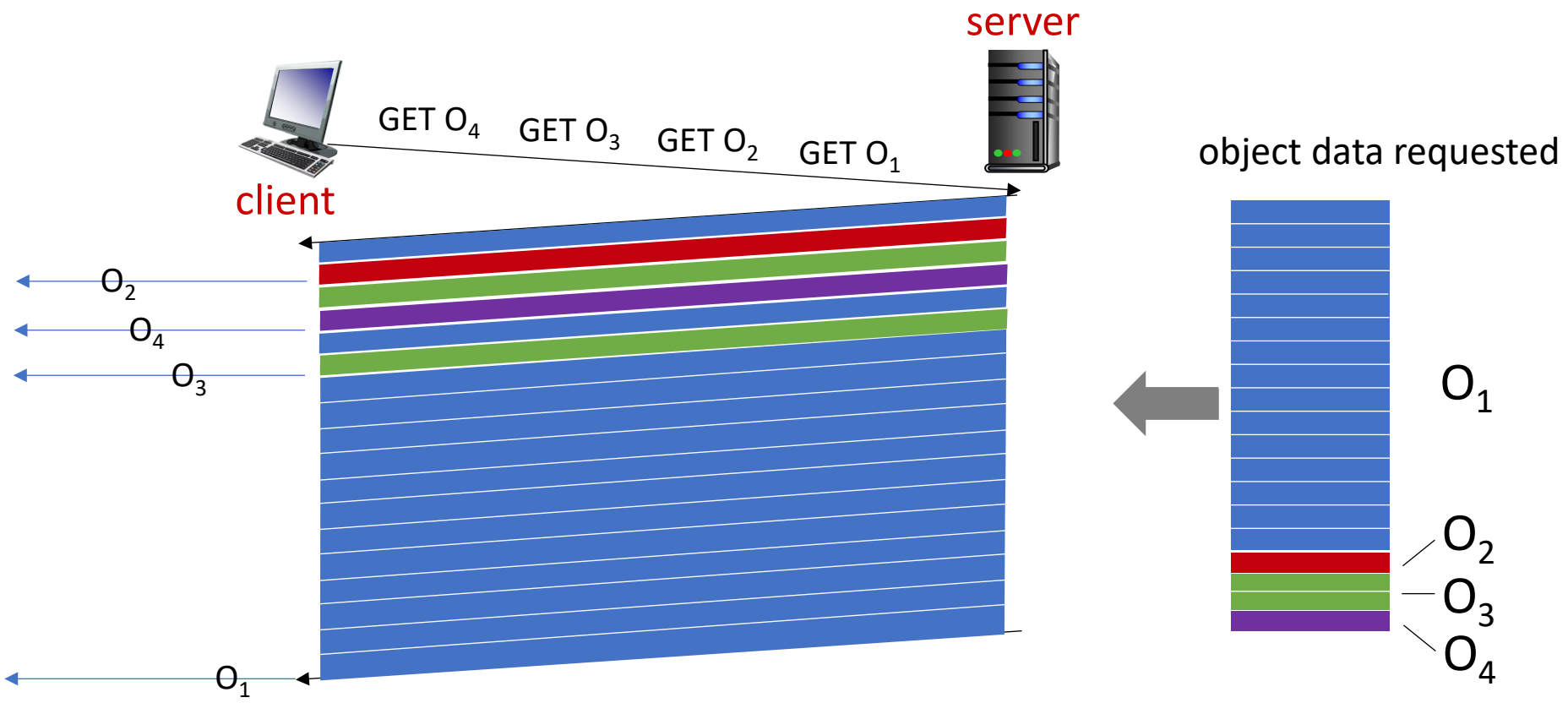


2.26 HTTP/2

- **Key goal**: decreased delay in multi-object HTTP requests
- HTTP1.1: introduced multiple, pipelined GETs over single TCP connection
  - server responds in-order (FCFS: first-come-first-served scheduling) to GET requests
  - with FCFS, small object may have to wait for transmission (head-of-line (HOL) blocking) behind large object(s)
  - loss recovery (retransmitting lost TCP segments) stalls object transmission
- HTTP/2: [RFC 7540, 2015] increased flexibility at server in sending objects to client:
  - methods, status codes, most header fields unchanged from HTTP 1.1
  - transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
  - push unrequested objects to client
  - divide objects into frames, schedule frames to mitigate HOL blocking

2.27 HTTP/2: Mitigating HOL Blocking

HTTP/2: objects divided into frames, frame transmission interleaved



O_2, O_3, O_4 delivered quickly, O_1 slightly delayed

2.28 HTTP/2 to HTTP/3

HTTP/2 over single TCP connection means:

- recovery from packet loss still stalls all object transmissions
  - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- HTTP/3: adds security, per object error- and congestion-control (more pipelining) over UDP
  - more on HTTP/3 in transport layer.

Week 3 - Agenda

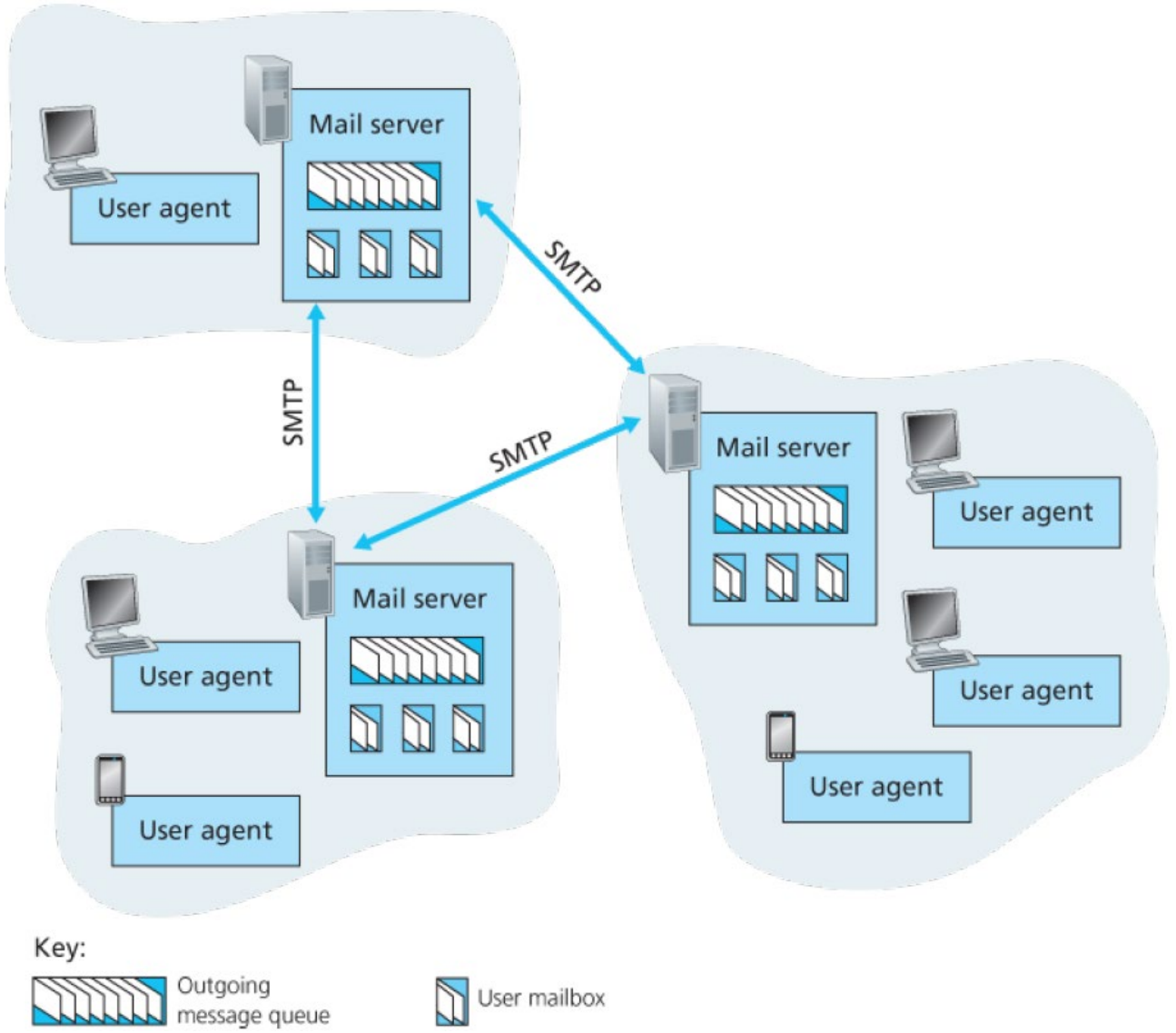
02 The Web and HTTP

03 Electronic Mail in the Internet

04 Next Week

3.1 Electronic Mail on the Internet

- Electronic mail has been around since the beginning of the Internet.
- It was the most popular application when the Internet was in its infancy (tr bebeklik), and has become more elaborate and powerful over the years.
- It remains one of the Internet's most important and utilized applications.
- Three major components: **user agents**, **mail servers**, and the **Simple Mail Transfer Protocol (SMTP)**.
- Microsoft Outlook and Apple Mail are examples of user agents for e-mail.
- Mail servers form the core of the e-mail infrastructure. Each recipient has a mailbox located in one of the mail servers.
- SMTP is the principal application-layer protocol for Internet electronic mail.



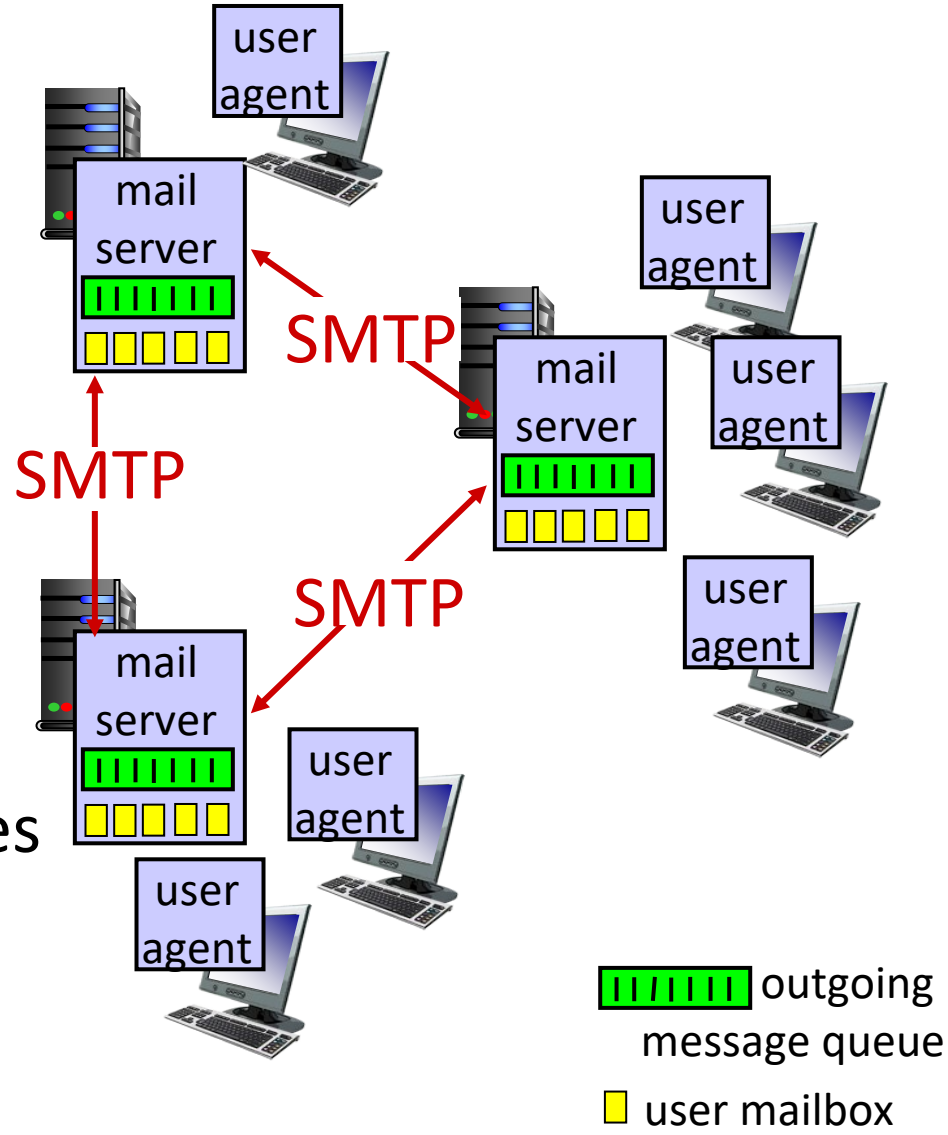
3.2 E-mail

Three major components:

- user agents
- mail servers
- simple mail transfer protocol: SMTP

User Agent

- a.k.a. “mail reader”
- composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client
- outgoing, incoming messages stored on server



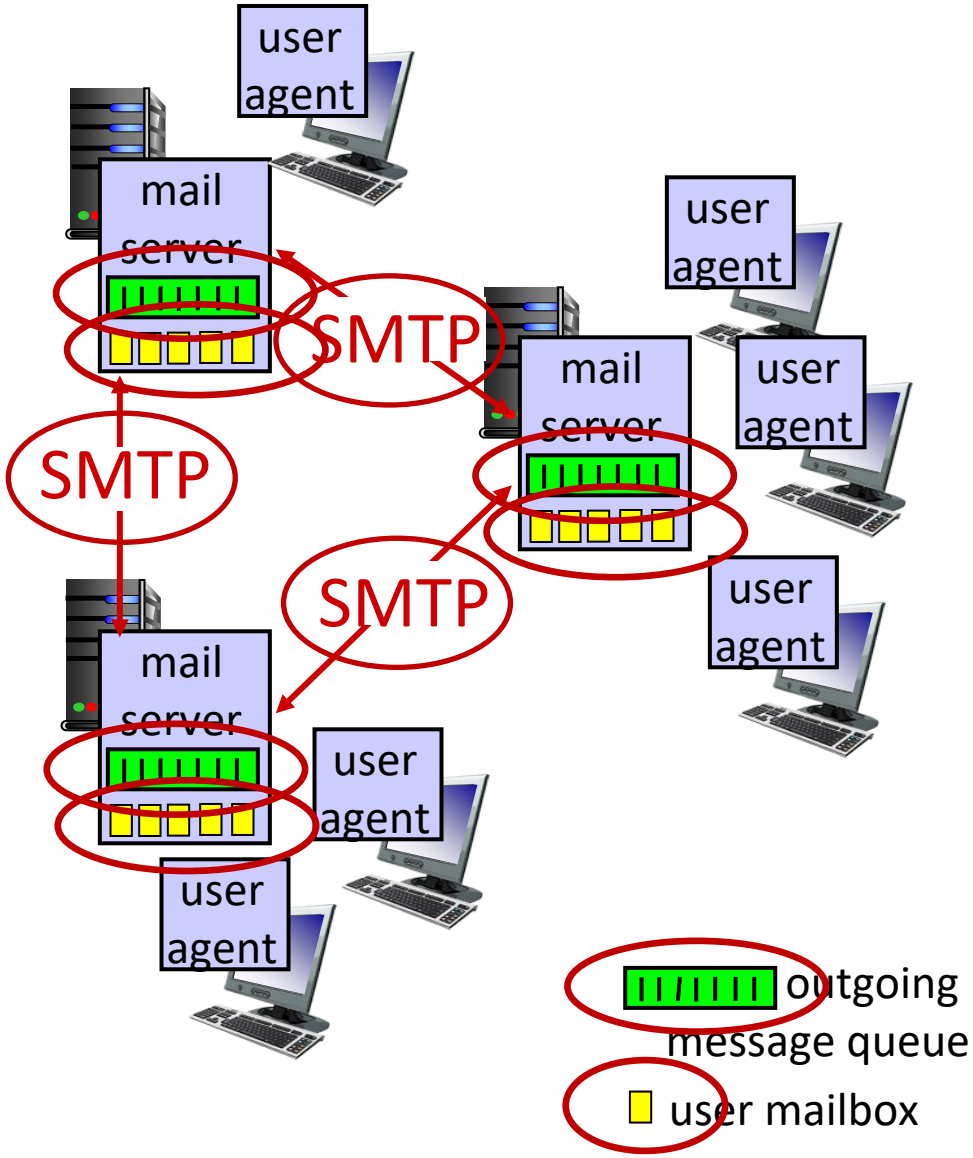
3.3 Mail Servers

mail servers:

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages

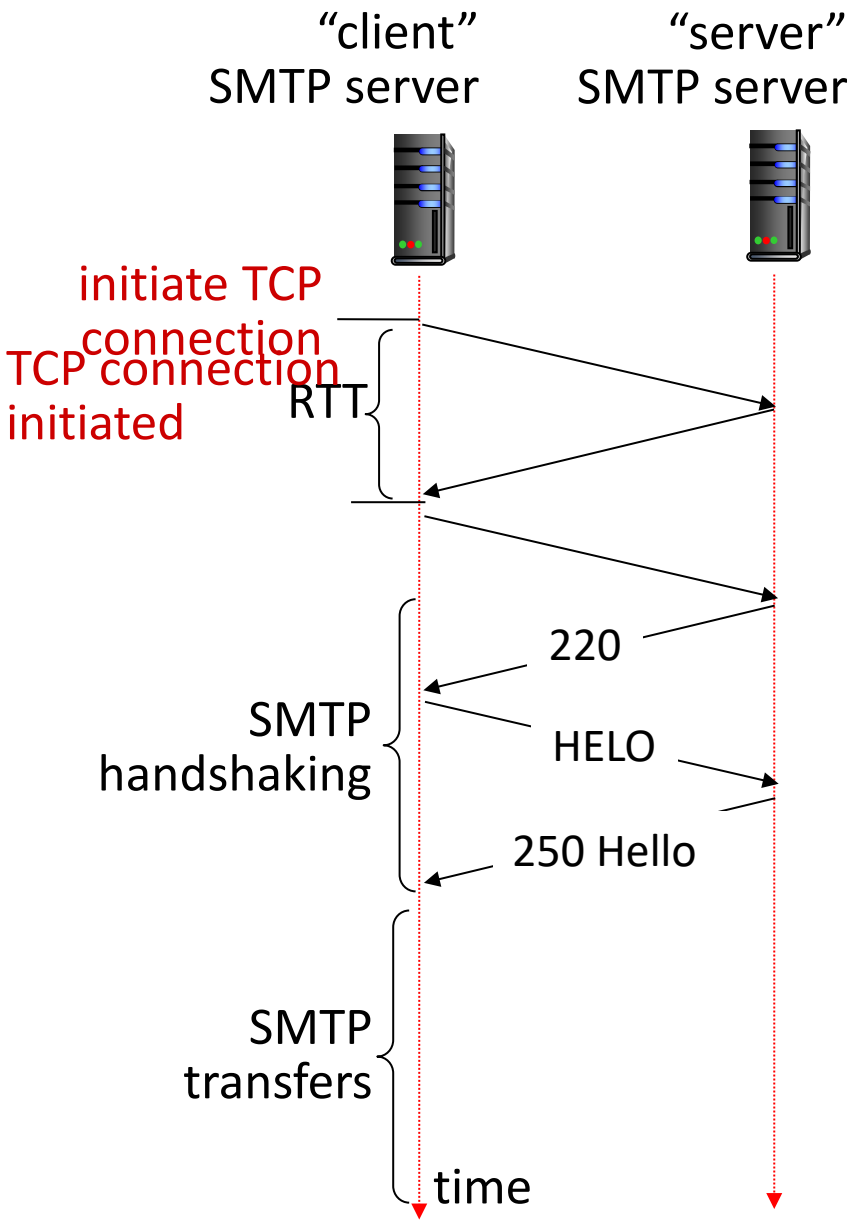
SMTP protocol between mail servers to send email messages

- client: sending mail server
- "server": receiving mail server



3.4 SMTP

- uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
  - direct transfer: sending server (acting like client) to receiving server
- three phases of transfer
  - SMTP handshaking (greeting)
  - SMTP transfer of messages
  - SMTP closure
- command/response interaction (like HTTP)
  - commands: ASCII text
  - response: status code and phrase



3.5 Scenario: Alice Sends E-mail to Bob

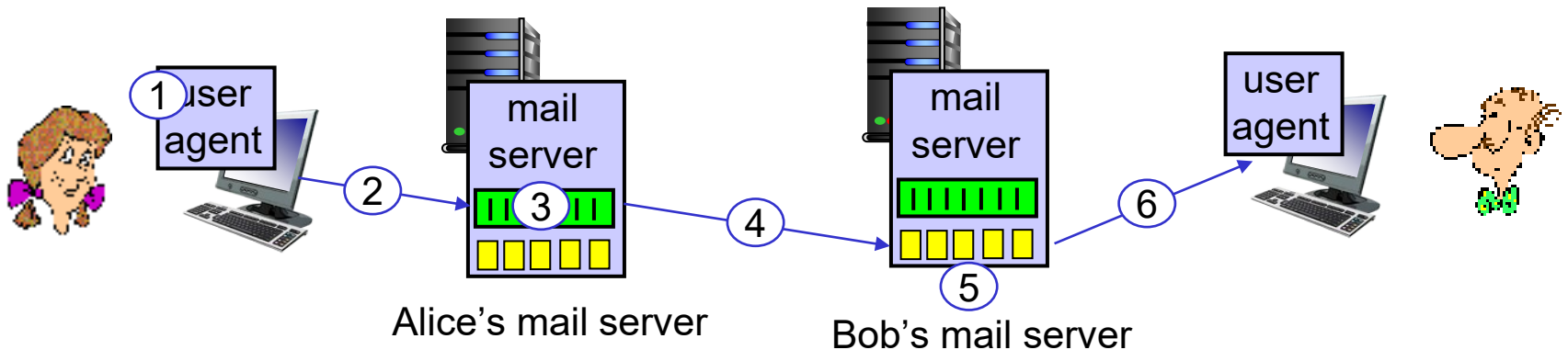
1) Alice uses UA to compose e-mail message “to” bob@someschool.edu

2) Alice’s UA sends message to her mail server using SMTP; message placed in message queue

3) client side of SMTP at mail server opens TCP connection with Bob’s mail server

5) Bob’s mail server places the message in Bob’s mailbox

6) Bob invokes his user agent to read message



3.6 Sample SMTP interaction

```
S: 220 hamburger.edu
```

3.7 Comparison with HTTP

- HTTP: client pull
- SMTP: client push
- both have ASCII command/response interaction, status codes
- HTTP: each object encapsulated in its own response message
- SMTP: multiple objects sent in multipart message
- SMTP uses persistent connections
- SMTP requires message (header & body) to be in 7-bit ASCII
- SMTP server uses CRLF.CRLF to determine end of message (carriage return, line feed)

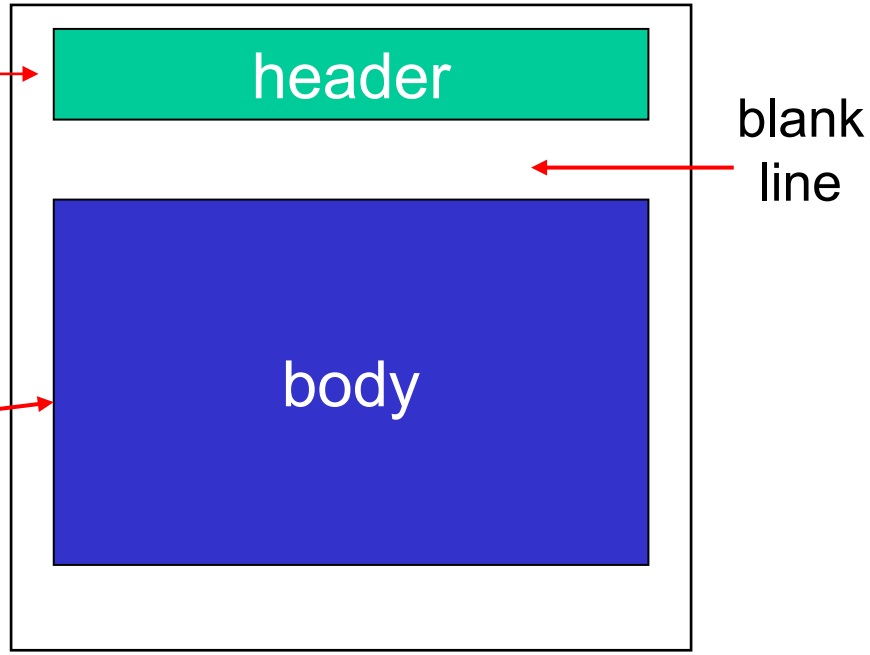
3.8 Mail Message Format

SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 7231 defines HTTP)

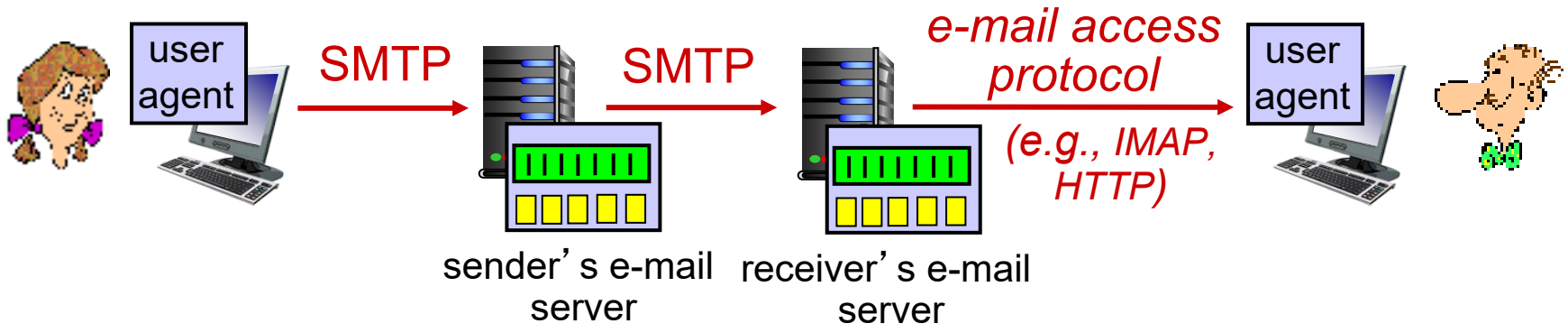
RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

- header lines, e.g.,
  - To:
  - From:
  - Subject:

  these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the "message" , ASCII characters only



3.9 Retrieving Email: Mail Access Protocols



- SMTP: delivery/storage of e-mail messages to receiver's server
- mail access protocol: retrieval from server
  - IMAP: Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- HTTP: gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of STMP (to send), IMAP (or POP) to retrieve e-mail messages

3.10 POP3 (Post Office Protocol 3)

- POP3 is an extremely simple mail access protocol. It is defined in [RFC 1939], which is short and quite readable.
- Because the protocol is so simple, its functionality is rather limited.
- POP3 begins when the user agent (the client) opens a TCP connection to the mail server (the server) on port **110**. With the TCP connection established, POP3 progresses through three phases: authorization, transaction, and update.
- During the first phase, **authorization**, the user agent sends a username and a password (in the clear) to authenticate the user.
- During the second phase, **transaction**, the user agent retrieves messages; also during this phase, the user agent can mark messages for deletion, remove deletion marks, and obtain mail statistics.
- The third phase, **update**, occurs after the client has issued the quit command, ending the POP3 session; at this time, the mail server deletes the messages that were marked for deletion.
- There are two possible responses:
  - **+OK** (sometimes followed by server-to-client data), used by the server to indicate that the previous command was fine; and
  - **-ERR** , used by the server to indicate that something was wrong with the previous command.

3.11 IMAP (Internet Messaging Access Protocol)

- With POP3 access, once Bob has downloaded his messages to the local machine, he can create mail folders and move the downloaded messages into the folders. Bob can then delete messages, move messages across folders, and search for messages (by sender name or subject).
- But this paradigm poses a problem for the user, who would prefer to maintain a folder hierarchy on a remote server that can be accessed from any computer. This is not possible with POP3.
- **IMAP** enables users to organize messages into folders, flag messages for urgency or follow-up, and save draft messages on the server. Users can also have multiple email client applications that sync with the email server to consistently show which messages have been read or are still unread.
- An IMAP server listens on port number **143**, while IMAP over Secure Sockets Layer (SSL)/Transport Layer Security is assigned port number **993**.

3.12 IMAP Operation

These are the general steps and processes involved in an IMAP operation:

- When a user signs into the email client -- e.g., Microsoft Outlook -- the client contacts the server using IMAP.
- The connection is made on a specific port.
- The headers of all emails are displayed by the email client.
- IMAP only downloads a message to the client when the user clicks on it; attachments are not automatically downloaded.
- Users can check their messages much more quickly with IMAP than with other email retrieval protocols, like Post Office Protocol 3 (POP3).
- Email messages remain on the server unless the user explicitly deletes them.

3.13 SMTP vs POP3 vs IMAP

- SMTP: message submission/relay and delivery between mail servers.
- POP3: retrieves messages, traditionally emphasizing download-oriented access.
- IMAP: accesses and synchronizes messages and folders maintained on the server.
- Webmail uses HTTP/HTTPS between the browser and webmail service; the provider uses mail protocols behind the interface.


Any Questions

4.1 Next Lecture (26 Oct.!!!)

- We will focus on chapters 2: dns, p2p, streaming, etc.

Your duties:

✓ Review this lecture note via learn.khas.edu.tr

✓ Read chapter 2 from Kurose & Ross's textbook.



Thank You
For Your Attention!

